M07 — Observabilidad y resiliencia con ASP.NET Core 8 · Teoría

Autor del curso: JAIME GALLO

Cómo estudiar el módulo

Los cuatro puntos conservan diez bloques cada uno. Cada bloque desarrolla su concepto, explica un ejemplo con referencias a sus líneas físicas y añade una ampliación aplicada a AceriaApp: fundamento, caso, límites y comprobación. Los doce diagramas, tres por punto, se distribuyen en SVG vectorial y PNG. Las líneas vacías cuentan para numerar los ejemplos, aunque no necesitan una explicación; cada cita **Línea N** ocupa una viñeta separada.

Punto de partida. Se continúa desde `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`, con 29 proyectos, 18 proyectos de pruebas, 283 pruebas y 34 documentos en docs. Permanecen .NET 8, EF Core 8 y SQL Server Express/LocalDB, las API de Producción e Inventario, el cliente HTTP, RabbitMQ y los mecanismos de outbox e inbox. La lectura de stock sigue siendo informativa: no reserva ni descuenta existencias.

Recorrido. El punto 7.1 añade Serilog JSON y correlación segura entre procesos. El 7.2 separa salud del proceso y acceso a dependencias. El 7.3 normaliza fallos inesperados con IExceptionHandler y ProblemDetails, conservando Result y sus códigos de negocio. El 7.4 aplica Polly v8 únicamente al cliente de lectura de Inventario, con límites de intentos, tiempo y circuito. La integración técnica compartida reside en `AceriaApp.Observabilidad`; el dominio no depende de ASP.NET, Serilog ni Polly.

Convenciones. Los fragmentos son ejemplos independientes. Algunos utilizan contratos conceptuales para centrar la explicación; los archivos completos, paquetes fijados, namespaces, rutas y pruebas están en las prácticas. Los ejemplos de Polly usan `Microsoft.Extensions.Http.Resilience` y namespaces de Polly; los de salud usan `Microsoft.Extensions.Diagnostics.HealthChecks` y `Microsoft.AspNetCore.Diagnostics.HealthChecks`. El fragmento AddDbContextCheck requiere su integración específica de EF Core. El tiempo de estudio depende de instalación, ensayos y diagnóstico; no se promete ejecutar el material ampliado en veintisiete minutos.

Alcance y evidencia. El laboratorio no despliega un backend de logs, métricas ni tracing distribuido completo. Los catálogos de análisis se conservan y se acompañan de comportamiento real. Las pruebas controladas verifican secuencias exactas; los recorridos entre procesos y las dependencias SQL/AMQP reales verifican su integración. Una configuración declarada, una simulación o un test omitido no acreditan una capacidad ejecutada. Los contenidos terminan con el cierre de M7.

Alcance de los logs seguros. Las comprobaciones de privacidad se aplican a los nuevos eventos propios de M7: resumen HTTP, manejo de excepciones y callbacks de Polly. Excluyen rutas crudas, query strings, cuerpos, Authorization y mensajes libres de excepciones; un enricher elimina RequestPath heredado de los scopes. Los workers de M6 y EF conservan logging heredado y no quedan acreditados como auditados integralmente por esas pruebas.

Punto 7.1 — Logging

Módulo 7: Observabilidad y resiliencia

Objetivos de aprendizaje:

- Comprender el papel del logging en sistemas distribuidos.
- Identificar los niveles de log y su uso adecuado.
- Comprender la diferencia entre logging estructurado y logging no estructurado.
- Comprender la integración de Serilog con ASP.NET Core.
- Aplicar los conceptos al comportamiento real de AceriaApp y contrastar sus resultados con pruebas.

Desarrollo teórico

Bloque 1 — Definición de logging

El logging es el proceso de registrar eventos que ocurren durante la ejecución de un sistema. Cada evento se registra con un nivel, un mensaje y un conjunto de propiedades. Los eventos se almacenan en un destino que puede ser la consola, un archivo, una base de datos o un servicio externo. El logging permite reconstruir lo que ocurrió en el sistema, diagnosticar errores y auditar el comportamiento. En sistemas distribuidos, el logging es más complejo porque los eventos ocurren en varios procesos. La correlación entre eventos de distintos servicios es esencial para diagnosticar problemas que atraviesan varios procesos.

```
Logging en sistemas distribuidos:

Servicio A → evento → destino de logs.
Servicio B → evento → destino de logs.
Servicio C → evento → destino de logs.
Correlación → identificador común.
```

- **Línea 3:** `Servicio A → evento → destino de logs.` → primer servicio.
- **Línea 4:** `Servicio B → evento → destino de logs.` → segundo servicio.
- **Línea 5:** `Servicio C → evento → destino de logs.` → tercer servicio.
- **Línea 6:** `Correlación → identificador común.` → identificador para correlacionar.

El logging se caracteriza por tres propiedades. La primera es que registra eventos; pueden ser texto libre o eventos estructurados, siendo estos últimos preferibles para análisis automático. La segunda es que asocia cada evento con un nivel. La tercera es que permite la correlación entre servicios. Estas propiedades aportan evidencia de ejecución y ayudan al diagnóstico; no garantizan por sí solas observabilidad completa ni auditoría duradera.

La aplicación del logging tiene dos consecuencias prácticas. La primera es que los errores se diagnostican con información precisa. La segunda es que el comportamiento del sistema se audita. La combinación de ambas permite que el sistema sea mantenible y que los problemas se resuelvan con rapidez.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un evento útil responde a una pregunta concreta: qué operación se intentó, en qué servicio, con qué resultado y en cuánto tiempo. Logging es una de las señales de observabilidad. Los mensajes no sustituyen métricas agregadas ni una traza distribuida; tampoco constituyen por sí solos un registro de auditoría con garantías de conservación.

Ejemplo razonado. Al consultar CHAPA-3MM, Producción genera un resumen de petición y propaga un identificador a Inventario. Dos eventos con el mismo CorrelationId permiten relacionar las peticiones, aunque cada proceso escriba en su propia consola JSON. Servicio y marca temporal evitan confundir emisores.

Riesgo, alternativa y límite. Registrar cada instrucción produce volumen sin explicar mejor el negocio. Un log de «inicio» sin resultado puede corresponder a éxito, cancelación o caída del proceso. Prioriza un evento final con campos estables y conserva los eventos de negocio que aportan información diferente.

Comprobación y aprendizaje. Ejecuta una lectura correcta y otra con Inventario detenido. Localiza los resúmenes por correlación y compara estado y duración. Si solo sabes que se llamó a un método, falta evidencia del resultado; añade información al límite de la operación, no mensajes indiscriminados.

Bloque 2 — Niveles de log

Los niveles de log clasifican los eventos según su gravedad. Los niveles habituales de eventos son seis: Trace, Debug, Information, Warning, Error y Critical. El nivel Trace se usa para información muy detallada que normalmente solo se activa en diagnóstico. El nivel Debug se usa para información de depuración. El nivel Information se usa para información de operación normal. El nivel Warning se usa para situaciones anómalas que no impiden la operación. El nivel Error se usa para errores que impiden una operación. El nivel Critical se usa para errores que afectan a todo el sistema.

```
Niveles de log:

Trace       → información muy detallada.
Debug       → información de depuración.
Information → operación normal.
Warning     → situación anómala.
Error       → error que impide una operación.
Critical    → error que afecta a todo el sistema.
```

- **Línea 3:** `Trace → información muy detallada.` → información muy detallada.
- **Línea 4:** `Debug → información de depuración.` → información de depuración.
- **Línea 5:** `Information → operación normal.` → operación normal.
- **Línea 6:** `Warning → situación anómala.` → situación anómala.
- **Línea 7:** `Error → error que impide una operación.` → error que impide una operación.
- **Línea 8:** `Critical → error que afecta a todo el sistema.` → error que afecta a todo el sistema.

Los niveles se caracterizan por tres propiedades. La primera es que cada nivel tiene un uso específico. La segunda es que el nivel mínimo se configura por entorno. La tercera es que los niveles permiten filtrar los eventos. Las tres propiedades permiten que el logging sea útil sin generar ruido excesivo.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Microsoft.Extensions.Logging define seis niveles de eventos, desde Trace hasta Critical, además del valor None que desactiva la emisión. Serilog los representa como Verbose, Debug, Information, Warning, Error y Fatal. El adaptador traduce los nombres: no hay que crear una segunda escala propia.

Ejemplo razonado. Una consulta que termina correctamente admite Information. Un fallo transitorio recuperado puede producir Warning. Una petición que termina en error inesperado corresponde a Error. Critical se reserva para una condición grave que impide continuar el servicio, no para cualquier 500 individual.

Riesgo, alternativa y límite. Un 404 esperado no significa necesariamente un incidente. Elevar todos los fallos funcionales a Error vuelve inservibles los filtros. Tampoco conviertas el nivel en una regla de negocio: cambiar Information a Warning no debe modificar la respuesta ni los efectos de la operación.

Comprobación y aprendizaje. Con nivel mínimo Warning, verifica que se ocultan los eventos Information sin alterar el comportamiento HTTP. Después vuelve a Information y comprueba que reaparecen. La prueba separa filtrado de logs de ejecución del caso de uso.

Bloque 3 — Logging estructurado

El logging estructurado registra eventos con una plantilla y propiedades tipadas, en lugar de depender únicamente de una cadena de texto ya construida. Cada evento se registra como un objeto con propiedades nombradas. Las propiedades permiten filtrar y agregar los eventos de forma automática. El logging estructurado se serializa habitualmente en JSON. La serialización permite que las herramientas de análisis procesen los eventos sin expresiones regulares.

```
logger.LogInformation(
    "Orden {OrdenId} creada con {CantidadLineas} líneas",
    orden.Id,
    orden.Lineas.Count);
```

- **Línea 1:** `logger.LogInformation(` → registra un evento de nivel Information.
- **Línea 2:** `"Orden {OrdenId} creada con {CantidadLineas} líneas",` → mantiene una plantilla estable con dos propiedades.
- **Línea 3:** `orden.Id,` → aporta la identidad técnica de la orden.
- **Línea 4:** `orden.Lineas.Count);` → aporta un contador y evita datos personales del cliente.

El logging estructurado se caracteriza por tres propiedades. La primera es que las propiedades son tipadas. La segunda es que las propiedades se pueden filtrar. La tercera es que los eventos pueden serializarse en JSON conservando sus propiedades. Las tres propiedades permiten que el análisis de logs sea automático y preciso.

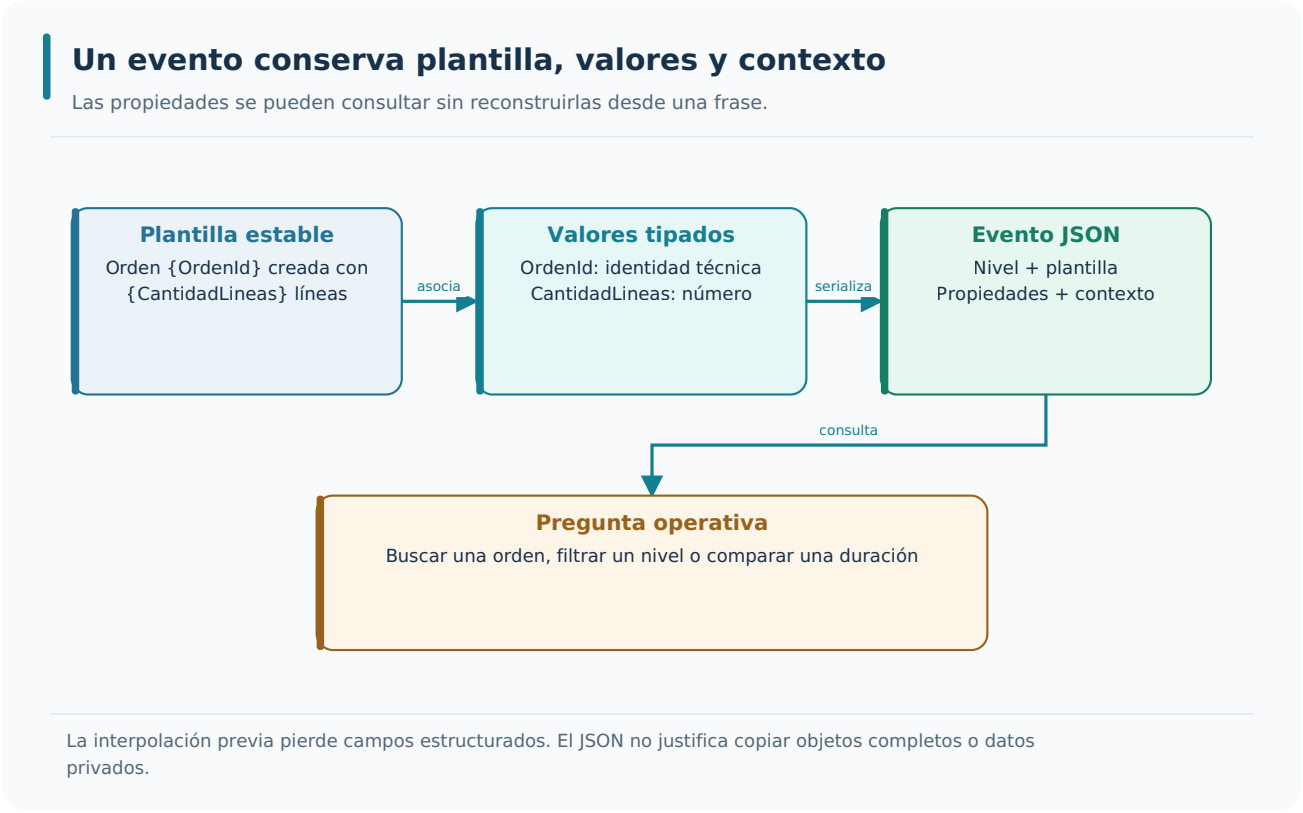
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La plantilla es estable y los valores viajan separados. El nombre OrdenId se convierte en una propiedad consultable; interpolar una cadena antes de llamar al logger pierde esa separación. El JSON es una representación posible de los eventos, no la definición del logging estructurado.

Ejemplo razonado. Un evento «Orden {OrdenId} planificada con {CantidadLineas} líneas» permite buscar una orden o agrupar por número de líneas sin analizar la redacción. El identificador técnico se ha elegido deliberadamente; el nombre o dirección del cliente no son necesarios para esa pregunta.

Riesgo, alternativa y límite. La desestructuración de un objeto completo puede arrastrar datos privados, navegaciones de EF o cientos de propiedades. Usa una lista explícita de campos. Conservar tipos también importa: una duración numérica se puede agregar; una cadena como «tardó mucho» no aporta una medida.

Comprobación y aprendizaje. Parsea como JSON los eventos propios nuevos de M7 y verifica propiedades y tipos. Prueba caracteres especiales para confirmar que el serializador los escapa. Busca un dato sensible centinela en todos los eventos de esa categoría; el ensayo no acredita una auditoría de los workers heredados de M6 ni de todo EF.



Interpretación del gráfico: Una plantilla estable conserva propiedades tipadas; el texto interpolado ya no distingue sus valores como campos.

Bloque 4 — Serilog como biblioteca de logging

Serilog es una biblioteca de logging estructurado para .NET. Serilog se integra con ASP.NET Core mediante el paquete `Serilog.AspNetCore`. Serilog permite configurar sinks, enrichers y formatos. Los sinks son destinos de los eventos: consola, archivo, Seq, Elasticsearch. Los enrichers añaden propiedades a los eventos: nombre de la máquina, identificador de la petición. Los formatos determinan la serialización de los eventos: JSON, texto, compacto. La configuración se realiza en el arranque de la aplicación.

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .Enrich.FromLogContext()
    .WriteTo.Console(new JsonFormatter())
    .CreateLogger();
```

- **Línea 1:** `Log.Logger = new LoggerConfiguration()` → crea la configuración.
- **Línea 2:** `.MinimumLevel.Information()` → establece el nivel mínimo.
- **Línea 3:** `.Enrich.FromLogContext()` → añade propiedades del contexto.
- **Línea 4:** `.WriteTo.Console(new JsonFormatter())` → escribe en consola en JSON.
- **Línea 5:** `.CreateLogger();` → crea el logger.

Serilog se caracteriza por tres propiedades. La primera es que es una biblioteca de logging estructurado. La segunda es que permite configurar sinks y enrichers. La tercera es que se integra con ASP.NET Core. Las tres propiedades permiten que el logging sea flexible y estructurado.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un sink determina dónde se escribe; un enricher añade contexto; un formatter decide cómo se representa. Son responsabilidades diferentes. El curso fija Serilog.AspNetCore 8.0.3 y escribe JSON en consola para que ambos procesos puedan observarse sin instalar un servidor de análisis adicional.

Ejemplo razonado. LogContext añade CorrelationId durante el alcance de una petición. Una propiedad Servicio distingue Produccion de Inventario. JsonFormatter conserva valores con nombres, de modo que un lector puede reunir las líneas de ambas consolas sin cambiar las plantillas del código.

Riesgo, alternativa y límite. Un archivo local requiere rotación, retención, permisos y tratamiento de disco lleno. El ejemplo de sink de archivo sirve como alternativa conceptual; el recorrido entregado utiliza consola. Añadir otro sink no garantiza que los eventos se entreguen de forma duradera ni que sobrevivan a cualquier caída.

Comprobación y aprendizaje. Arranca la API, realiza una petición y parsea cada línea emitida. Verifica que no se mezclan mensajes de texto libre con los eventos esperados. Si dos proveedores escriben el mismo evento, corrige su registro en el arranque antes de añadir filtros que oculten el duplicado.

Bloque 5 — Integración de Serilog con ASP.NET Core

La integración de Serilog con ASP.NET Core se realiza con el paquete `Serilog.AspNetCore`. El paquete ofrece el método `UseSerilog` que se aplica al `IHostBuilder`. El método reemplaza el logger por defecto de ASP.NET Core. La configuración se realiza en el arranque. La integración permite que los logs de ASP.NET Core y los logs de la aplicación se escriban en el mismo destino.

```
var builder = WebApplication.CreateBuilder(args);

builder.Host.UseSerilog((context, config) =>
{
    config
        .MinimumLevel.Information()
        .Enrich.FromLogContext()
        .WriteTo.Console(new JsonFormatter());
});
```

- **Línea 1:** `var builder = WebApplication.CreateBuilder(args);` → crea el constructor.
- **Línea 3:** `builder.Host.UseSerilog((context, config) =>` → configura Serilog en el host.
- **Línea 4:** `{` → abre el bloque de configuración.
- **Línea 5:** `config` → comienza la configuración.
- **Línea 6:** `.MinimumLevel.Information()` → establece el nivel mínimo.
- **Línea 7:** `.Enrich.FromLogContext()` → añade propiedades del contexto.
- **Línea 8:** `.WriteTo.Console(new JsonFormatter());` → escribe en consola en JSON.
- **Línea 9:** `});` → cierra el bloque.

La integración se caracteriza por tres propiedades. La primera es que reemplaza el logger por defecto. La segunda es que unifica los logs de ASP.NET Core y de la aplicación. La tercera es que permite configurar sinks y enrichers. Las tres propiedades permiten que el logging sea coherente en todo el sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La aplicación consume `ILogger<T>`; el arranque conecta esa abstracción a Serilog. El dominio no necesita conocer sinks ni HttpContext. La biblioteca AceriaApp.Observabilidad concentra la integración técnica de ambas API, mientras cada host conserva su nombre de servicio y su configuración.

Ejemplo razonado. El orden de middleware permite establecer correlación antes de registrar el resumen y de gestionar una excepción. El resumen observa el resultado final del flujo. En las prácticas se registra método, plantilla de ruta, estado y duración, evitando copiar URL, query string o cuerpo.

Riesgo, alternativa y límite. Los scopes de ASP.NET pueden añadir RequestPath aunque la plantilla propia no lo solicite. CamposHttpSeguros elimina esa propiedad después de FromLogContext. También se limitan categorías automáticas, pero los workers de M6 y los eventos heredados de EF conservan su logging: no se afirma una auditoría integral de esas fuentes.

Comprobación y aprendizaje. Envía una petición con un parámetro centinela y una cabecera Authorization de prueba. Revisa los nuevos resúmenes HTTP de ambas API: deben conservar correlación y resultado, y omitir ambos valores y RequestPath. Verifica también los eventos propios de errores y Polly; el alcance de la comprobación debe quedar explícito.

Bloque 6 — Correlación de logs

La correlación de logs permite vincular eventos de distintos servicios que pertenecen a la misma operación. La correlación se implementa con un identificador único que se propaga entre servicios. El identificador se incluye en las cabeceras HTTP o en los mensajes del broker. El identificador se añade a cada evento de log. La correlación permite relacionar los eventos registrados de una operación, sin garantizar una traza distribuida completa.

```
app.Use(async (context, next) =>
{
    var values = context.Request.Headers["X-Correlation-ID"];
    var candidate = values.Count == 1 ? values[0] : null;
    var valid = candidate is { Length: >= 1 and <= 64 }
        && candidate.All(c => char.IsAsciiLetterOrDigit(c) || c is '-' or '_');
    var id = valid ? candidate! : Guid.NewGuid().ToString("N");
    context.Items["CorrelationId"] = id;
    context.Response.Headers["X-Correlation-ID"] = id;
    context.Response.OnStarting(() =>
    {
        context.Response.Headers["X-Correlation-ID"] = id;
        return Task.CompletedTask;
    });
    using (LogContext.PushProperty("CorrelationId", id))
    {
        await next();
    }
});
```

- **Línea 1:** `app.Use(async (context, next) =>` → registra el middleware antes del flujo que necesita correlación.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `var values = context.Request.Headers["X-Correlation-ID"];` → lee todos los valores de la cabecera.
- **Línea 4:** `var candidate = values.Count == 1 ? values[0] : null;` → acepta como candidata una única entrada.
- **Línea 5:** `var valid = candidate is { Length: >= 1 and <= 64 }` → limita la longitud.

- **Línea 6:** `&& candidate.All(c => char.IsAsciiLetterOrDigit(c) || c is '-' or '_');` → permite solo caracteres ASCII definidos.
- **Línea 7:** `var id = valid ? candidate! : Guid.NewGuid().ToString("N");` → sustituye una entrada inválida por un identificador generado.
- **Línea 8:** `context.Items["CorrelationId"] = id;` → deja disponible el valor validado para componentes posteriores.
- **Línea 9:** `context.Response.Headers["X-Correlation-ID"] = id;` → establece el identificador efectivo.
- **Línea 10:** `context.Response.OnStarting(() =>` → registra una acción justo antes de enviar cabeceras.
- **Línea 11:** `{` → abre la acción de respuesta.
- **Línea 12:** `context.Response.Headers["X-Correlation-ID"] = id;` → restituye la cabecera si el manejador de errores limpió la respuesta.
- **Línea 13:** `return Task.CompletedTask;` → termina la acción sin trabajo asíncrono adicional.
- **Línea 14:** `});` → cierra el registro de OnStarting.
- **Línea 15:** `using (LogContext.PushProperty("CorrelationId", id))` → crea un alcance de logging que se libera al terminar.
- **Línea 16:** `{` → abre el alcance.
- **Línea 17:** `await next();` → continúa la petición sin bloquear el hilo.
- **Línea 18:** `}` → cierra el alcance.
- **Línea 19:** `});` → termina el registro del middleware.

La correlación se caracteriza por tres propiedades. La primera es que el identificador se propaga entre servicios. La segunda es que se añade a cada evento. La tercera es que permite agrupar los eventos disponibles de la operación. Las tres propiedades permiten que el diagnóstico sea preciso en sistemas distribuidos.

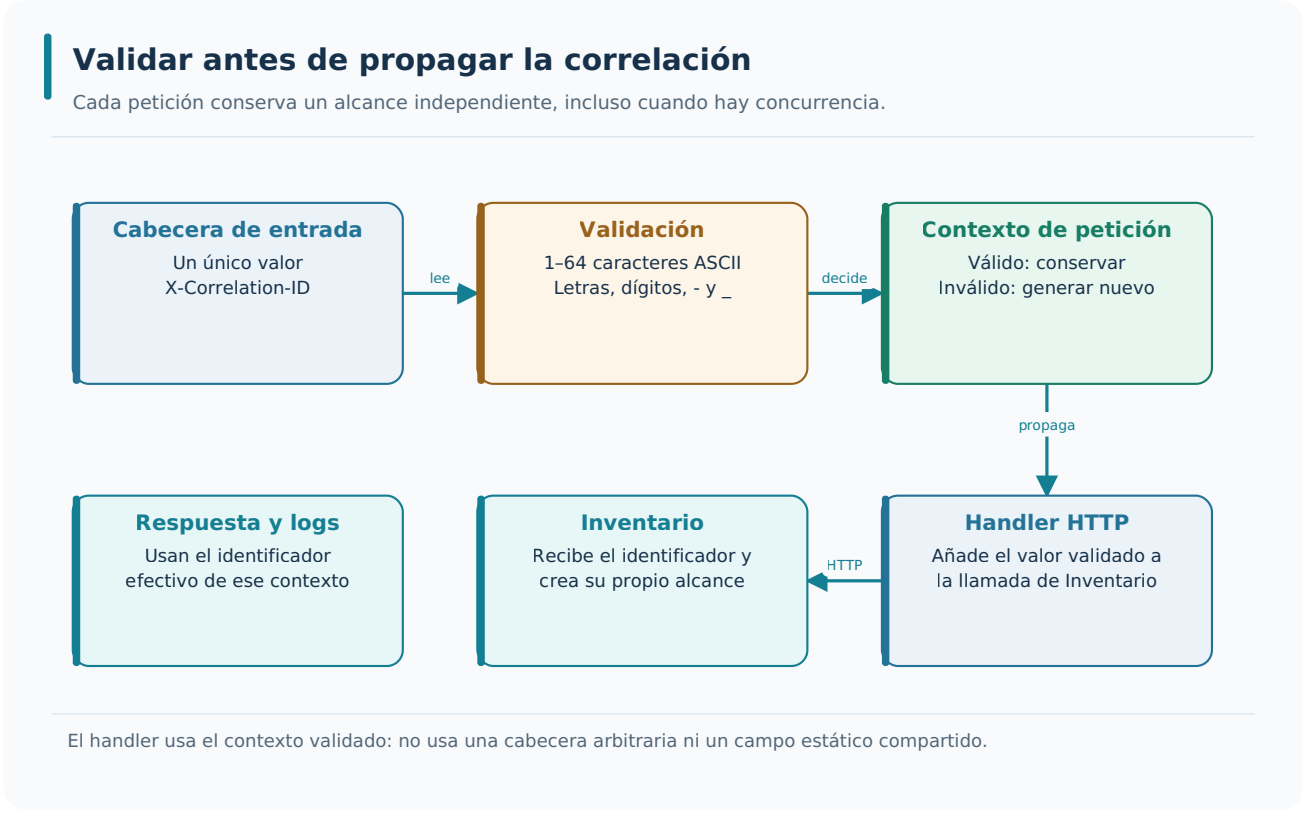
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un identificador recibido por red es entrada no confiable. El middleware acepta exactamente un valor de entre 1 y 64 caracteres ASCII alfanuméricos, guion o subrayado. Si falta o no cumple, genera un valor nuevo; la respuesta y la llamada saliente usan el valor ya validado.

Ejemplo razonado. Una petición con X-Correlation-ID: lote-42 conserva ese valor en la respuesta y en la llamada a Inventario. OnStarting vuelve a escribir la cabecera justo antes de enviarla, incluso si el manejador de excepciones limpió las cabeceras. Una petición paralela con lote-43 mantiene su propio contexto.

Riesgo, alternativa y límite. CorrelationId agrupa eventos, pero no prueba identidad del cliente ni autorización. Tampoco sustituye TraceId y SpanId de Activity o un sistema de tracing. El contexto de una petición no debe quedar retenido después de terminar; using delimita correctamente las propiedades de LogContext.

Comprobación y aprendizaje. Prueba valor válido, ausente, demasiado largo, varios valores y caracteres no permitidos. Comprueba que un valor rechazado no aparece ni en logs ni en la petición remota. Ejecuta solicitudes concurrentes y verifica que los grupos de eventos no intercambian identificadores.



Interpretación del gráfico: La entrada se valida antes de llegar al contexto, la respuesta y el handler HTTP saliente. El alcance de cada petición mantiene aisladas las solicitudes concurrentes.

Bloque 7 — Logging de errores

El logging de errores registra los errores con la información necesaria para diagnosticarlos. La información puede incluir tipo, mensaje, pila y contexto, pero debe seleccionarse según sensibilidad. En este curso se registra el tipo y contexto permitido, sin valores libres de excepciones. El logging de errores se realiza en el manejador global de excepciones. El manejador captura la excepción, registra el evento y devuelve una respuesta al cliente. El logging de errores no sustituye la gestión de errores: la complementa.

```
try
{
    await next();
}
catch (Exception ex) when (ex is not OperationCanceledException)
{
    Log.Error("Fallo no controlado de tipo {ExceptionType}", ex.GetType().Name);
    throw;
}
```

- **Línea 1:** `try` → inicia el bloque ilustrativo.
- **Línea 2:** `{` → abre la región protegida.
- **Línea 3:** `await next();` → ejecuta el siguiente componente.
- **Línea 4:** `}` → cierra la región.
- **Línea 5:** `catch (Exception ex) when (ex is not OperationCanceledException)` → evita clasificar aquí la cancelación como fallo inesperado.
- **Línea 6:** `{` → abre el tratamiento.

- **Línea 7:** `Log.Error("Fallo no controlado de tipo {ExceptionType}", ex.GetType().Name);` → registra el tipo sin el mensaje ni la excepción completa.
- **Línea 8:** `throw;` → conserva la pila al propagar el error.
- **Línea 9:** `}` → termina el tratamiento; el manejador global completo aparece en las prácticas.

El logging de errores se caracteriza por tres propiedades. La primera es que registra información suficiente y permitida del fallo. La segunda es que añade propiedades del contexto. La tercera es que complementa la gestión de errores. Las tres propiedades permiten que el diagnóstico sea preciso.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Registrar una excepción completa puede incluir credenciales, SQL, rutas o información del usuario en Message, Data e InnerException. El laboratorio registra tipo de fallo y contexto técnico permitido. Una captura amplia solo tiene sentido en el límite que decide cómo responder o limpiar recursos.

Ejemplo razonado. Si una consulta genera una excepción con un mensaje centinela, el evento puede indicar ExceptionType = InvalidOperationException y conservar CorrelationId. El cliente recibe un error genérico. El equipo localiza el incidente por la correlación sin devolver ni publicar el texto arbitrario.

Riesgo, alternativa y límite. Capturar, registrar y relanzar en cada capa produce varios eventos para un único fallo. Establece un responsable del registro final. Si se relanza, throw conserva la pila; throw ex la altera. La cancelación solicitada por el usuario no se clasifica automáticamente como fallo inesperado.

Comprobación y aprendizaje. Fuerza una excepción controlada y cuenta los eventos del error. Busca el centinela tanto en JSON como en la respuesta HTTP. A continuación cancela una petición lenta: la ausencia de un 500 fabricado demuestra que se diferencia el abandono del cliente de un fallo interno.

Bloque 8 — Buenas prácticas de logging

Las buenas prácticas de logging son cinco. La primera es usar logging estructurado en lugar de cadenas concatenadas. La segunda es usar plantillas con propiedades nombradas. La tercera es no registrar información sensible. La cuarta es configurar el nivel mínimo por entorno. La quinta es correlacionar los eventos con un identificador común. La aplicación de las cinco prácticas produce logs útiles y seguros.

```
Buenas prácticas de logging:

1. Logging estructurado.
2. Plantillas con propiedades.
3. Sin información sensible.
4. Nivel mínimo por entorno.
5. Correlación con identificador.
```

- **Línea 3:** `1. Logging estructurado.` → primera práctica.
- **Línea 4:** `2. Plantillas con propiedades.` → segunda práctica.
- **Línea 5:** `3. Sin información sensible.` → tercera práctica.
- **Línea 6:** `4. Nivel mínimo por entorno.` → cuarta práctica.
- **Línea 7:** `5. Correlación con identificador.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen logs útiles. La segunda es que protegen la información sensible. La tercera es que permiten el análisis automático. Las tres propiedades permiten que el logging sea un activo del sistema.

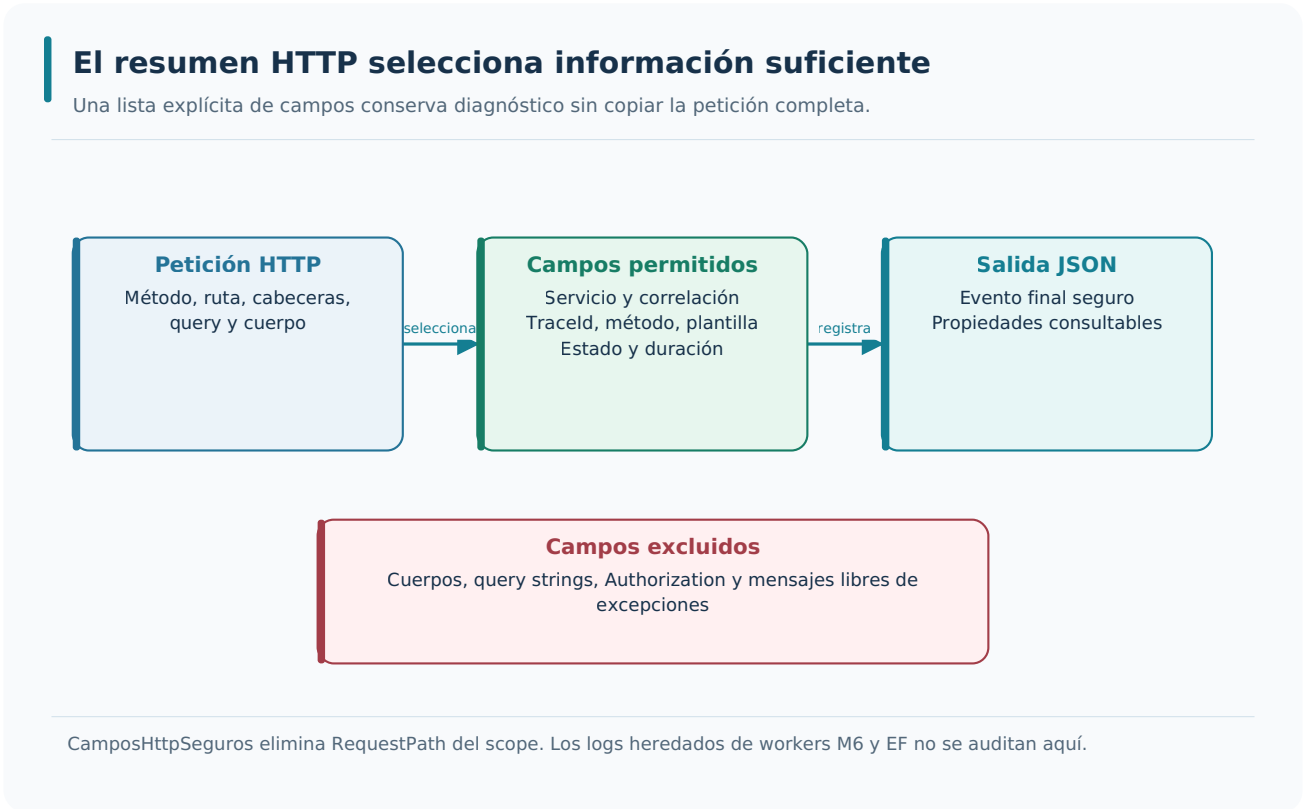
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La selección de campos debe ser positiva: una lista de propiedades necesarias resulta más auditable que serializar todo y quitar algunas. Para las peticiones se usan Servicio, CorrelationId, TraceId, método, plantilla de ruta, estado y duración. Para mensajería se conservan EventoId y OrdenId donde proceda.

Ejemplo razonado. La plantilla /api/materiales/{codigo} agrupa lecturas sin almacenar cada URL concreta. La duración numérica permite localizar lentitud. El estado HTTP distingue ausencia, indisponibilidad y éxito; la respuesta completa no es necesaria para hacer esa clasificación.

Riesgo, alternativa y límite. Hash de un dato sensible no equivale automáticamente a anonimización. Tampoco una cabecera que «normalmente es inocua» puede copiarse sin validar. Evita cuerpos, query strings, Authorization y valores arbitrarios de excepciones. La correlación admitida no cambia estas reglas.

Comprobación y aprendizaje. Introduce un centinela diferente en cada canal de entrada y busca los valores en los nuevos resúmenes HTTP, eventos propios de excepción y callbacks de Polly. Comprueba que mantienen los campos necesarios para diagnosticar. El logging heredado de workers M6 y EF exige otra revisión; esta prueba no acredita su privacidad completa.



Interpretación del gráfico: Los nuevos eventos propios de M7 seleccionan campos operativos y excluyen datos libres y RequestPath heredado del scope. No se afirma una auditoría completa de workers M6 o EF.

Bloque 9 — Configuración por entorno

La configuración de logging se ajusta por entorno. En desarrollo se usa el nivel Debug para obtener información detallada. En producción se usa el nivel Information para evitar el ruido. En pruebas se usa el nivel Warning para reducir la salida. La configuración se realiza en el archivo `appsettings.json` o en el código de arranque. La configuración por entorno permite que el logging se adapte a cada contexto.

```
{
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.Hosting.Lifetime": "Information"
      }
    }
  }
}
```

- **Línea 1:** `{` → abre el objeto raíz.
- **Línea 2:** `"Serilog": {` → abre la sección de Serilog.
- **Línea 3:** `"MinimumLevel": {` → abre la sección de nivel mínimo.
- **Línea 4:** `"Default": "Information",` → nivel por defecto.
- **Línea 5:** `"Override": {` → abre la sección de sobrescritura.
- **Línea 6:** `"Microsoft": "Warning",` → nivel para Microsoft.
- **Línea 7:** `"Microsoft.Hosting.Lifetime": "Information"` → nivel para el ciclo de vida.
- **Línea 8:** `}` → cierra la sección de sobrescritura.
- **Línea 9:** `}` → cierra la sección de nivel mínimo.
- **Línea 10:** `}` → cierra la sección de Serilog.
- **Línea 11:** `}` → cierra el objeto raíz.

La configuración por entorno se caracteriza por tres propiedades. La primera es que el nivel se ajusta por entorno. La segunda es que la configuración se documenta. La tercera es que la configuración se versiona. Las tres propiedades permiten que el logging se adapte a cada contexto.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Los niveles dependen del objetivo operativo; Debug en desarrollo o Warning en pruebas son ejemplos, no obligaciones universales. En un test de logging debes habilitar el nivel que pretendes verificar. La configuración declarada solo tiene efecto si el arranque la lee o establece los valores equivalentes.

Ejemplo razonado. El componente AddLoggingAceria recibe servicio y configuración. En el laboratorio, Observabilidad:NivelMinimo usa Information por defecto y appsettings.Production.json establece Warning. Un diagnóstico temporal puede cambiar el nivel y volver después al habitual; el cambio no modifica contratos HTTP ni permite capturar información privada.

Riesgo, alternativa y límite. ReadFrom.Configuration requiere la integración y los paquetes correspondientes; escribir una sección Serilog no configura mágicamente el logger. Mantén los secretos fuera de archivos versionados y documenta qué valores acepta el componente de registro de este curso.

Comprobación y aprendizaje. Arranca con dos configuraciones conocidas y observa un evento que debe filtrarse en una de ellas. Verifica también que el resumen final sigue presente en la configuración de referencia. Contrastar el archivo y la salida evita confundir intención de configuración con configuración aplicada.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica el logging en el módulo 7. Serilog se integra con ASP.NET Core. El nivel mínimo se configura por entorno. Los logs del recorrido se escriben en consola JSON; el archivo es una alternativa de sink que requiere gestión adicional. Los eventos se correlacionan con un identificador común. La información sensible no se registra. La validación conserva el test de coherencia y añade pruebas de JSON, correlación, aislamiento concurrente, privacidad y propagación HTTP entre procesos.

```
Aplicación al proyecto del curso:

Biblioteca     → Serilog.
Sinks          → consola JSON.
Enrichers      → LogContext.
Correlación    → X-Correlation-Id.
Nivel          → configurado por entorno.
Validación     → coherencia y comportamiento real.
```

- **Línea 3:** `Biblioteca → Serilog.` → Serilog.
- **Línea 4:** `Sinks → consola JSON.` → consola JSON.
- **Línea 5:** `Enrichers → LogContext.` → LogContext.
- **Línea 6:** `Correlación → X-Correlation-Id.` → X-Correlation-Id.
- **Línea 7:** `Nivel → configurado por entorno.` → configurado por entorno.
- **Línea 8:** `Validación → coherencia y comportamiento real.` → coherencia y comportamiento real.

La aplicación del logging prepara el proyecto para los puntos 7.2, 7.3 y 7.4. El punto 7.2 estudiará los health checks. El punto 7.3 estudiará la gestión de excepciones. El punto 7.4 estudiará el retry y el circuit breaker mediante Polly. El punto 7.1 establece la base que se aplicará en los tres puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El catálogo de conceptos ayuda a discutir decisiones, pero la observabilidad se acredita con eventos emitidos por procesos reales. El módulo conserva ese análisis y lo acompaña de middleware, propagación HTTP y pruebas de salida JSON. Una validación de cadenas no prueba esas capacidades.

Ejemplo razonado. La lectura desde Producción hasta Inventario deja dos resúmenes relacionados y una cabecera de respuesta. Los eventos de outbox e inbox heredados continúan aportando identidades de mensajes; el módulo no añade un backend de trazas ni promete una traza distribuida completa de todo el broker.

Riesgo, alternativa y límite. Un test unitario de un formateador puede pasar mientras el host olvida registrar el middleware. Por eso las pruebas combinan componentes aislados y un recorrido HTTP entre dos procesos. Los fallos de infraestructura se verifican con servicios reales cuando el escenario depende de ellos.

Comprobación y aprendizaje. Compara la evidencia de una prueba de configuración con la del recorrido real: la primera confirma una regla, la segunda el cableado y la emisión. Explica qué afirma cada una. Esa distinción se reutiliza al evaluar salud, errores y resiliencia en los puntos siguientes.

Resumen rápido de la teoría

- El logging registra eventos que ocurren durante la ejecución del sistema.
- Los niveles de log son Trace, Debug, Information, Warning, Error y Critical.
- El logging estructurado usa propiedades tipadas en lugar de cadenas.

- Serilog es una biblioteca de logging estructurado para .NET.
- La integración de Serilog con ASP.NET Core se realiza con `UseSerilog`.
- La correlación de logs vincula eventos de distintos servicios con un identificador común.
- El logging de errores selecciona información técnica permitida y contexto correlacionado, evitando detalles sensibles.
- Las buenas prácticas son logging estructurado, plantillas con propiedades, sin información sensible, nivel por entorno y correlación.
- La configuración por entorno ajusta el nivel mínimo según el contexto.

Punto 7.2 — Health Checks

Módulo 7: Observabilidad y resiliencia

Objetivos de aprendizaje:

- Comprender qué son los health checks y su propósito en sistemas distribuidos.
- Identificar los tipos de health checks: liveness, readiness y startup.
- Comprender el papel de los health checks en orquestadores y balanceadores.
- Comprender la integración de health checks con ASP.NET Core.
- Aplicar los conceptos al comportamiento real de AceriaApp y contrastar sus resultados con pruebas.

Desarrollo teórico

Bloque 1 — Definición de health check

Un health check es un punto de entrada que expone el estado del servicio. El health check devuelve un código HTTP que resume el resultado de las comprobaciones seleccionadas para esa ruta. Los health checks se consultan desde orquestadores, balanceadores y sistemas de monitorización. El health check no reemplaza al logging: lo complementa. El consumidor puede tratar la respuesta HTTP como éxito o fallo; el modelo interno de ASP.NET Core distingue Healthy, Degraded y Unhealthy. La señal se usa para decidir si el servicio recibe tráfico o si debe reiniciarse.

```
Health check:

Orquestador → consulta health check → Servicio.
Respuesta: 200 OK → disponible.
Respuesta: 503 Service Unavailable → no disponible.
```

- **Línea 3:** `Orquestador → consulta health check → Servicio.` → el orquestador consulta el estado.
- **Línea 4:** `Respuesta: 200 OK → disponible.` → el servicio está disponible.
- **Línea 5:** `Respuesta: 503 Service Unavailable → no disponible.` → el servicio no está disponible.

El health check se caracteriza por tres propiedades. La primera es que expone el estado del servicio. La segunda es que se consulta de forma periódica. La tercera es que permite traducir el estado interno a una decisión operativa. Las tres propiedades permiten que los sistemas externos tomen decisiones sobre el servicio.

La aplicación de health checks tiene dos consecuencias prácticas. La primera es que los servicios no disponibles se detectan de forma automática. La segunda es que una plataforma configurada puede

reiniciar procesos o retirar instancias del balanceador sin intervención manual. La combinación de ambas permite que el sistema sea más resiliente.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un health check responde a una pregunta operativa delimitada, no certifica que toda la aplicación funcione. El resultado es una observación puntual con un contrato HTTP. La plataforma consumidora decide qué hacer con esa señal; exponer una ruta por sí solo no reinicia ni retira instancias.

Ejemplo razonado. En AceriaApp, /health/live confirma que el proceso puede responder. /health/ready comprueba acceso a SQL Server y, cuando se habilitan los trabajadores, a RabbitMQ. Si SQL cae, el proceso sigue vivo aunque temporalmente no esté listo para el conjunto de operaciones que necesita persistencia.

Riesgo, alternativa y límite. Un endpoint que siempre responde Healthy no representa el estado de las dependencias. El extremo contrario, consultar todo en cada sonda, añade carga y puede crear una cadena de falsas alarmas. Define las dependencias imprescindibles y evita ejecutar cambios de negocio desde una sonda.

Comprobación y aprendizaje. Detén SQL Server y consulta ambas rutas. Readiness debe fallar y liveness permanecer disponible. Reinicia la dependencia y comprueba la recuperación. El resultado enseña a distinguir un proceso operativo de una operación de negocio realizable.

Bloque 2 — Tipos de health check

Los health checks se clasifican en tres tipos: liveness, readiness y startup. El liveness check indica si el servicio está vivo. Si el liveness check falla, el orquestador reinicia el servicio. El readiness check indica si el servicio está listo para recibir tráfico. Si el readiness check falla, el orquestador retira el servicio del balanceador. El startup check indica si el servicio ha terminado su arranque. Mientras la sonda de startup no supera su umbral, la plataforma permite el arranque y mantiene desactivadas las otras sondas; si agota el umbral configurado, puede reiniciar el contenedor.

```
Tipos de health check:

Liveness  → el servicio está vivo. Si falla, se reinicia.
Readiness → el servicio está listo. Si falla, se retira del balanceador.
Startup   → margen de arranque; al agotar el umbral puede reiniciarse.
```

- **Línea 3:** `Liveness → el servicio está vivo. Si falla, se reinicia.` → el servicio está vivo.
- **Línea 4:** `Readiness → el servicio está listo. Si falla, se retira del balanceador.` → el servicio está listo.
- **Línea 5:** `Startup → margen de arranque; al agotar el umbral puede reiniciarse.` → el servicio ha arrancado.

Los tres tipos se caracterizan por tres propiedades. La primera es que cada tipo tiene un propósito específico. La segunda es que cada tipo se consulta en un momento distinto. La tercera es que cada tipo provoca una decisión distinta, aunque los ejecute el mismo orquestador. Las tres propiedades permiten que el sistema detecte problemas de forma precisa.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Liveness pregunta si conviene reiniciar este proceso; readiness pregunta si conviene enviarle tráfico ahora; startup permite un arranque prolongado antes de activar las otras evaluaciones. Los umbrales pertenecen a la plataforma. Una sonda de startup que supera su límite de fallos también puede provocar reinicio.

Ejemplo razonado. Una migración o inicialización lenta puede necesitar protección de arranque. Una caída compartida de SQL no se arregla reiniciando todas las API. AceriaApp expone live y ready; startup se estudia como concepto operativo y no se presenta como despliegue Kubernetes incluido.

Riesgo, alternativa y límite. Usar la misma lista de comprobaciones para live y ready destruye su diferencia, aunque las URLs sean distintas. Tampoco cada sonda tiene necesariamente un consumidor diferente: un mismo orquestador puede ejecutar las tres con políticas distintas.

Comprobación y aprendizaje. Escribe la acción esperada ante tres situaciones: proceso bloqueado, SQL inaccesible y arranque aún en curso. Comprueba que la ruta elegida comunica la causa adecuada. Si todas las respuestas proponen reiniciar, el diseño probablemente está mezclando responsabilidades.

Tres preguntas operativas diferentes

Los nombres de las rutas no sustituyen la selección de comprobaciones y sus umbrales.



En el curso se implementan live y ready. Una caída de SQL degrada ready sin hacer fallar live.

Interpretación del gráfico: Una dependencia caída afecta readiness. Liveness permanece independiente y startup concede margen limitado durante el arranque.

Bloque 3 — Health checks y disponibilidad

Los health checks aportan una señal de disponibilidad de las comprobaciones seleccionadas; no bastan para medir toda la disponibilidad funcional del servicio. Un servicio disponible responde con 200 a los health checks. Un servicio no disponible responde con 503 o no responde. La proporción de sondas exitosas sobre el total de consultas mide el éxito de esas sondas, durante el periodo y desde el observador elegidos. Los health checks permiten calcular la disponibilidad de forma automática. La disponibilidad es una de las métricas fundamentales de la observabilidad.

```
Disponibilidad:

200 OK → servicio disponible.
503 Service Unavailable → servicio no disponible.
Sin respuesta → fallo de observación: proceso, red o timeout.
```

- **Línea 3:** `200 OK → servicio disponible.` → el servicio responde.

- **Línea 4:** `503 Service Unavailable → servicio no disponible.` → el servicio responde con error.
- **Línea 5:** `Sin respuesta → fallo de observación: proceso, red o timeout.` → no se obtuvo respuesta; la causa requiere diagnóstico.

La disponibilidad se caracteriza por tres propiedades. La primera es que se mide de forma automática. La segunda es que se expone con health checks. La tercera es que se usa para tomar decisiones. Las tres propiedades permiten que el sistema sea observable y gestionable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La fracción de sondas exitosas mide disponibilidad de esa sonda desde ese observador. La disponibilidad funcional requiere definir qué operaciones cuentan, qué resultados son válidos y durante qué periodo se mide. Un 200 de salud no garantiza que una orden concreta pueda planificarse.

Ejemplo razonado. Inventario puede responder a ready y mantener una proyección retrasada porque hay mensajes pendientes. El acceso a SQL y AMQP es real, pero no equivale a haber vaciado outbox o inbox ni a cumplir un objetivo de frescura. Salud, latencia y progreso son preguntas diferentes.

Riesgo, alternativa y límite. Ausencia de respuesta admite varias causas: red interrumpida, timeout, resolución de nombre o proceso detenido. Etiquetarla siempre como «servicio caído» oculta el lugar de observación. Un muestreo cada treinta segundos tampoco detecta necesariamente un fallo que dura solo cinco.

Comprobación y aprendizaje. Compara una serie de sondas con el resultado de peticiones de negocio durante un fallo controlado. Registra desde dónde se realiza cada medida. Explica por qué dos observadores pueden obtener resultados distintos sin que uno de los registros sea falso.

Bloque 4 — Health checks en ASP.NET Core

ASP.NET Core ofrece soporte nativo para health checks con el paquete `Microsoft.Extensions.Diagnostics.HealthChecks`. El paquete permite registrar comprobaciones y exponerlas en un endpoint. El método `AddHealthChecks` registra el servicio de health checks. El método `MapHealthChecks` expone el endpoint. Los health checks pueden comprobar la base de datos, el broker de mensajería, los servicios externos y cualquier otro recurso.

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AceriaDbContext>("database", tags: new[] { "ready" })
    .AddCheck("self", () => HealthCheckResult.Healthy(), tags: new[] { "live" });

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("live")
});
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

- **Línea 1:** `builder.Services.AddHealthChecks()` → registra el servicio de salud.
- **Línea 2:** `.AddDbContextCheck<AceriaDbContext>("database", tags: new[] { "ready" })` → etiqueta la comprobación de EF Core exclusivamente para readiness.
- **Línea 3:** `.AddCheck("self", () => HealthCheckResult.Healthy(), tags: new[] { "live" });` → registra una comprobación del propio proceso.
- **Línea 5:** `app.MapHealthChecks("/health/live", new HealthCheckOptions` → mapea la ruta de liveness.

- **Línea 6:** `{` → abre sus opciones.
- **Línea 7:** `Predicate = check => check.Tags.Contains("live")` → selecciona únicamente los checks con tag live.
- **Línea 8:** `});` → termina la configuración de liveness.
- **Línea 9:** `app.MapHealthChecks("/health/ready", new HealthCheckOptions` → mapea la ruta de readiness.
- **Línea 10:** `{` → abre sus opciones.
- **Línea 11:** `Predicate = check => check.Tags.Contains("ready")` → selecciona únicamente los checks con tag ready.
- **Línea 12:** `});` → termina la configuración de readiness.

La integración con ASP.NET Core se caracteriza por tres propiedades. La primera es que el soporte es nativo. La segunda es que las comprobaciones se registran en el contenedor. La tercera es que los endpoints se exponen con `MapHealthChecks`. Las tres propiedades permiten que los health checks se integren con la aplicación.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. AddHealthChecks registra comprobaciones; MapHealthChecks selecciona cuáles ejecuta una ruta. Los tags son etiquetas y Predicate las filtra. Sin ese filtro, dos endpoints ejecutan todas las comprobaciones registradas. La separación semántica debe quedar escrita en opciones, no únicamente en el nombre de la URL.

Ejemplo razonado. El ejemplo marca self como live y database como ready. AddDbContextCheck usa la integración de EF Core para comprobar conectividad y requiere Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore. En el recorrido funcional se usa una comprobación SQL específica con una consulta real, tiempo acotado y cancelación.

Riesgo, alternativa y límite. Conectividad no prueba que todas las tablas o permisos de negocio sean correctos. No ejecutes migraciones desde el health check. Además, el registro de un check no hace que el servidor empiece a servir HTTP: el host debe construir y ejecutar su pipeline.

Comprobación y aprendizaje. Revisa los predicados y ejecuta las dos rutas con SQL detenido. Si live falla por la base de datos, inspecciona primero la selección de checks. Comprueba después que ready realmente abre la conexión y consulta, en lugar de inspeccionar solo un objeto de configuración.

Bloque 5 — Health checks personalizados

Los health checks personalizados comprueban recursos o capacidades específicas del servicio, normalmente en infraestructura y sin modificar el dominio. Un health check personalizado implementa la interfaz `IHealthCheck`. El método `CheckHealthAsync` devuelve un `HealthCheckResult` con el estado. El estado puede ser `Healthy`, `Degraded` o `Unhealthy`. El `Degraded` indica que el servicio funciona pero con funcionalidad reducida. El `Unhealthy` indica que la comprobación no cumple su criterio de salud; no implica necesariamente que el proceso esté detenido.

```
public class RabbitMqHealthCheck : IHealthCheck
{
    private readonly IConnection _connection;

    public RabbitMqHealthCheck(IConnection connection)
    {
        _connection = connection;
    }

    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken ct)
    {
        return _connection.IsOpen
            ? Task.FromResult(HealthCheckResult.Healthy("RabbitMQ disponible"))
            : Task.FromResult(HealthCheckResult.Unhealthy("RabbitMQ no disponible"));
    }
}
```

- **Línea 1:** `public class RabbitMqHealthCheck : IHealthCheck` → health check personalizado.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `private readonly IConnection _connection;` → dependencia de la conexión.
- **Línea 5:** `public RabbitMqHealthCheck(IConnection connection)` → constructor.
- **Línea 6:** `{` → abre el cuerpo del constructor.
- **Línea 7:** `_connection = connection;` → asigna la conexión.
- **Línea 8:** `}` → cierra el constructor.
- **Línea 10:** `public Task<HealthCheckResult> CheckHealthAsync(` → método de comprobación.
- **Línea 11:** `HealthCheckContext context,` → contexto de la comprobación.
- **Línea 12:** `CancellationToken ct)` → token de cancelación.
- **Línea 13:** `{` → abre el cuerpo del método.
- **Línea 14:** `return _connection.IsOpen` → comprueba si la conexión está abierta.
- **Línea 15:** `? Task.FromResult(HealthCheckResult.Healthy("RabbitMQ disponible"))` → devuelve Healthy.
- **Línea 16:** `: Task.FromResult(HealthCheckResult.Unhealthy("RabbitMQ no disponible"));` → devuelve Unhealthy.
- **Línea 17:** `}` → cierra el método.
- **Línea 18:** `}` → cierra la clase.

El health check personalizado se caracteriza por tres propiedades. La primera es que comprueba un recurso específico. La segunda es que devuelve un estado específico. La tercera es que se registra en el contenedor. Las tres propiedades permiten que el sistema compruebe cualquier recurso.

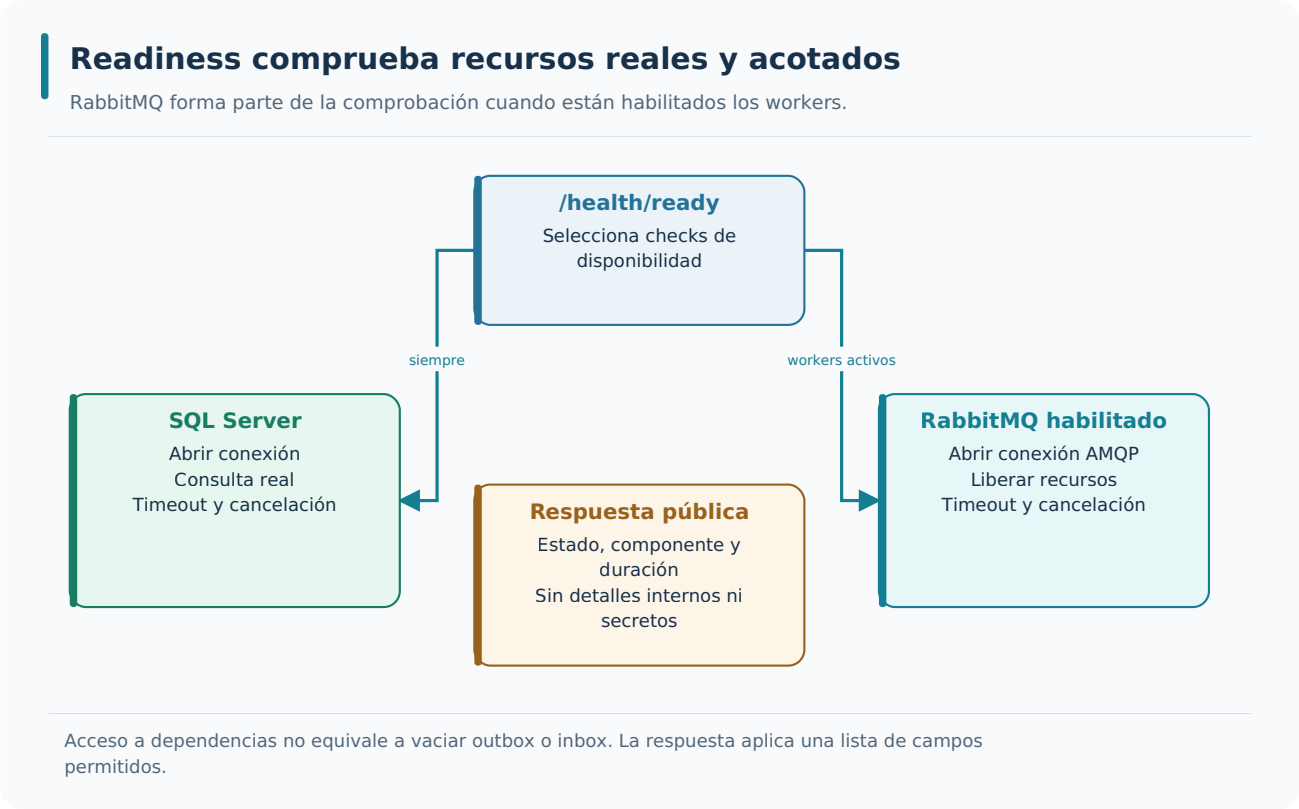
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un IHealthCheck personalizado implementa una prueba pequeña con un resultado interpretable. Para RabbitMQ, IsOpen de una conexión ya existente es una indicación local y puede tardar en reflejar un corte. La práctica realiza conexión AMQP con espera limitada, cancela cuando corresponde y libera recursos.

Ejemplo razonado. Con workers habilitados, una credencial de broker incorrecta impide readiness aunque la API siga respondiendo a live. Con workers deshabilitados, el broker no forma parte de esa comprobación. SQL sigue siendo una dependencia crítica de los casos de uso persistentes.

Riesgo, alternativa y límite. Abrir conexiones ilimitadas desde sondas muy frecuentes puede agravar la caída. Ajusta frecuencia, timeout y concurrencia; evita un retry largo dentro de la propia sonda. Healthy acredita la comprobación elegida, no permisos de todos los exchanges ni consumo correcto de cada mensaje.

Comprobación y aprendizaje. Ensaya broker sano, puerto inaccesible y credenciales no válidas en un entorno de prueba. Verifica duración acotada, estado no saludable y ausencia de credenciales en la respuesta. Comprueba que cancelar la petición no deja una operación esperando indefinidamente.



Interpretación del gráfico: Ready comprueba SQL y añade RabbitMQ cuando los workers están habilitados. Cada operación tiene espera limitada y cancelación.

Bloque 6 — Formato de la respuesta

La respuesta de un health check se puede formatear de varias formas. El formato por defecto es texto plano con el estado. El formato JSON se usa para que las herramientas procesen la respuesta. El formato personalizado se usa para adaptar la respuesta a un sistema específico. La elección del formato depende del consumidor del health check. Los orquestadores suelen usar el código HTTP. Las herramientas de monitorización usan el cuerpo de la respuesta.

```
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready"),
    ResponseWriter = async (context, report) =>
    {
        context.Response.ContentType = "application/json";
        var response = new
        {
            status = report.Status.ToString(),
            checks = report.Entries.Select(e => new
            {
                name = e.Key,
                status = e.Value.Status.ToString(),
                durationMs = e.Value.Duration.TotalMilliseconds
            })
        };
        await context.Response.WriteAsJsonAsync(response);
    }
});
```

- **Línea 1:** `app.MapHealthChecks("/health/ready", new HealthCheckOptions` → configura el endpoint.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `Predicate = check => check.Tags.Contains("ready"),` → selecciona comprobaciones de readiness.
- **Línea 4:** `ResponseWriter = async (context, report) =>` → configura el escritor de respuesta.
- **Línea 5:** `{` → abre el cuerpo del lambda.
- **Línea 6:** `context.Response.ContentType = "application/json";` → establece el tipo de contenido.
- **Línea 7:** `var response = new` → crea el objeto de respuesta.
- **Línea 8:** `{` → abre el cuerpo del objeto.
- **Línea 9:** `status = report.Status.ToString(),` → estado general.
- **Línea 10:** `checks = report.Entries.Select(e => new` → lista de comprobaciones.
- **Línea 11:** `{` → abre el cuerpo del objeto de comprobación.
- **Línea 12:** `name = e.Key,` → nombre de la comprobación.
- **Línea 13:** `status = e.Value.Status.ToString(),` → estado de la comprobación.
- **Línea 14:** `durationMs = e.Value.Duration.TotalMilliseconds` → duración numérica sin descripción interna ni excepción.
- **Línea 15:** `})` → cierra el objeto de comprobación.
- **Línea 16:** `};` → cierra el objeto de respuesta.
- **Línea 17:** `await context.Response.WriteAsJsonAsync(response);` → escribe la respuesta.
- **Línea 18:** `}` → cierra el lambda.
- **Línea 19:** `});` → cierra la configuración.

El formato de la respuesta se caracteriza por tres propiedades. La primera es que se adapta al consumidor. La segunda es que se documenta. La tercera es que se versiona con el resto de la API. Las tres propiedades permiten que el health check sea legible y procesable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La respuesta pública se construye con una lista permitida de campos: estado general, nombre y estado de cada componente, y duración. Las descripciones de checks y las excepciones pueden contener servidores, rutas o cadenas de conexión, por lo que no se serializan automáticamente.

Ejemplo razonado. Un consumidor puede ver database = Unhealthy y su duración sin conocer la contraseña ni el mensaje del proveedor SQL. El estado HTTP sigue siendo decisivo para un balanceador; el JSON añade detalle operativo limitado para una persona o un monitor.

Riesgo, alternativa y límite. HealthStatus admite Healthy, Degraded y Unhealthy. ASP.NET Core trata Healthy y Degraded como 200 por defecto y Unhealthy como 503. El laboratorio configura explícitamente Degraded y Unhealthy como 503 en sus rutas de salud: un estado degradado también deja de considerarse listo.

Comprobación y aprendizaje. Provoca una excepción que contenga un centinela secreto. Comprueba el código HTTP y el esquema del JSON, y busca el centinela en todos los campos. Verifica que la duración sea numérica y que una descripción interna no aparezca por haber serializado report.Entries directamente.

Bloque 7 — Health checks y contenedores

Los contenedores usan health checks para detectar servicios no disponibles. Docker define health checks en el `Dockerfile` con la instrucción `HEALTHCHECK`. Kubernetes define health checks con las directivas `livenessProbe`, `readinessProbe` y `startupProbe`. La acción depende de la plataforma: Docker HEALTHCHECK marca salud, mientras un orquestador puede reiniciar o retirar el contenedor según su política. La integración con contenedores permite que el sistema se recupere de forma automática.

```
Health checks en contenedores:

Dockerfile → HEALTHCHECK.
Kubernetes → livenessProbe, readinessProbe, startupProbe.
```

- **Línea 3:** `Dockerfile → HEALTHCHECK.` → health check en Docker.
- **Línea 4:** `Kubernetes → livenessProbe, readinessProbe, startupProbe.` → health checks en Kubernetes.

La integración con contenedores se caracteriza por tres propiedades. La primera es que los health checks se definen en la configuración del contenedor. La segunda es que el orquestador consulta los endpoints. La tercera es que las acciones configuradas pueden ejecutarse de forma automática. Las tres propiedades permiten que el sistema sea auto-recuperable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El contenedor empaqueta el proceso; el orquestador aplica una política sobre las sondas. Docker HEALTHCHECK marca un estado de salud, pero Docker por sí solo no reinicia automáticamente un contenedor solo por estar unhealthy. Kubernetes sí aplica acciones según el tipo de probe y sus umbrales.

Ejemplo razonado. La misma API de AceriaApp podría ejecutarse sin contenedores y conservar /health/live y /health/ready. Los tests del módulo invocan esas rutas y dependencias; la explicación de contenedores muestra cómo consumirlas, sin afirmar que se haya añadido un clúster al ejercicio.

Riesgo, alternativa y límite. El comando de comprobación necesita una herramienta presente en la imagen. Copiar un HEALTHCHECK que usa curl sin instalarlo informa de un fallo del comando, no de la API. Los intervalos y timeouts deben dejar margen al arranque y evitar cargas innecesarias.

Comprobación y aprendizaje. Para evaluar una configuración, identifica ruta, puerto, comando, timeout, periodo y umbral de fallos. Describe qué componente ejecuta el reinicio o retirada. Esta lectura evita atribuir a una respuesta 503 efectos que dependen de configuración externa.

Bloque 8 — Health checks y balanceadores

Los balanceadores usan health checks para decidir qué instancias reciben tráfico. El balanceador consulta el endpoint de readiness de cada instancia. Si la instancia responde con 200, el balanceador le envía tráfico. Si la instancia responde con 503, el balanceador la retira del grupo. La integración con balanceadores permite que el tráfico se distribuya solo entre instancias disponibles.

```
Balanceador y health checks:

Instancia 1 → 200 OK → recibe tráfico.
Instancia 2 → 503 → no recibe tráfico.
Instancia 3 → 200 OK → recibe tráfico.
```

- **Línea 3:** `Instancia 1 → 200 OK → recibe tráfico.` → primera instancia disponible.
- **Línea 4:** `Instancia 2 → 503 → no recibe tráfico.` → segunda instancia no disponible.
- **Línea 5:** `Instancia 3 → 200 OK → recibe tráfico.` → tercera instancia disponible.

La integración con balanceadores se caracteriza por tres propiedades. La primera es que el balanceador consulta periódicamente. La segunda es que el balanceador retira las instancias no disponibles. La tercera es que el tráfico se distribuye solo entre instancias disponibles. Las tres propiedades permiten que el sistema sea tolerante a fallos.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un balanceador decide por instancia y normalmente necesita varios resultados para retirarla o reincorporarla. La transición no es instantánea: hay periodo de consulta, umbral y peticiones que ya estaban en curso. Readiness reduce nuevas asignaciones, no cancela mágicamente todas las solicitudes existentes.

Ejemplo razonado. Si dos instancias de Producción usan SQL diferentes y una pierde acceso, la sonda puede retirar solo esa instancia. Si comparten un SQL detenido, ambas fallarán: la sonda hace visible una dependencia común, pero no crea una réplica de base de datos ni capacidad adicional.

Riesgo, alternativa y límite. Comprobar un servicio remoto no esencial en todas las instancias puede retirar toda la aplicación por una función secundaria. Antes de incluir una dependencia, pregunta qué operaciones deben dejar de recibir tráfico. En el curso se usa un criterio simple documentado, sin enrutamiento por capacidad.

Comprobación y aprendizaje. Razona qué ocurre entre la primera sonda fallida y la retirada efectiva. El cliente todavía necesita timeout y una respuesta coherente durante esa ventana. Salud y resiliencia se complementan porque operan a escalas y con responsabilidades diferentes.



Interpretación del gráfico: El balanceador usa umbrales para retirar o reincorporar instancias. Las peticiones en curso todavía necesitan su propio control de fallos.

Bloque 9 — Buenas prácticas de health checks

Las buenas prácticas de health checks son cinco. La primera es separar el liveness y el readiness en endpoints distintos. La segunda es comprobar solo los recursos críticos en el readiness. La tercera es usar el startup check para servicios con arranque lento. La cuarta es no exponer información sensible en la respuesta. La quinta es configurar los tiempos de espera y los intervalos de consulta. La aplicación de las cinco prácticas produce health checks útiles y seguros.

```
Buenas prácticas de health checks:

1. Endpoints separados para liveness y readiness.
2. Solo recursos críticos en readiness.
3. Startup check para arranques lentos.
4. Sin información sensible.
5. Tiempos e intervalos configurados.
```

- **Línea 3:** `1. Endpoints separados para liveness y readiness.` → primera práctica.
- **Línea 4:** `2. Solo recursos críticos en readiness.` → segunda práctica.
- **Línea 5:** `3. Startup check para arranques lentos.` → tercera práctica.
- **Línea 6:** `4. Sin información sensible.` → cuarta práctica.
- **Línea 7:** `5. Tiempos e intervalos configurados.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen health checks útiles. La segunda es que protegen la información sensible. La tercera es que permiten el diagnóstico. Las tres propiedades permiten que los health checks sean un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Diseña sondas baratas, sin efectos de negocio y con tiempo máximo. El token de cancelación debe llegar a las operaciones SQL y AMQP, y los recursos abiertos deben liberarse. Una configuración de timeout que no se propaga al trabajo real solo limita la espera del observador.

Ejemplo razonado. Una consulta mínima a SQL acredita que se abre una conexión y se recibe respuesta. Una conexión AMQP acredita acceso al broker. Ninguna sonda debe crear una orden, modificar stock ni publicar un evento para «ver si funciona»: esos efectos pertenecen a pruebas de integración controladas.

Riesgo, alternativa y límite. Las sondas pueden convertirse en ruido de logs o en una carga repetitiva. No registres secretos al explicar por qué falló una dependencia. Los endpoints públicos entregan un resumen acotado; los detalles de diagnóstico requieren su propio canal y control de acceso.

Comprobación y aprendizaje. Mide cuántas operaciones externas causa una llamada a ready y qué sucede al cancelarla. Repite con dependencias lentas y comprueba que el tiempo observado se mantiene dentro del margen esperado. Verifica además que los datos de negocio no han cambiado.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica health checks en el módulo 7. Los endpoints se separan en `/health/live` y `/health/ready`. El endpoint de liveness comprueba que la aplicación está en ejecución. El endpoint de readiness comprueba SQL Server con una consulta real y RabbitMQ cuando están habilitados los workers. La respuesta se formatea en JSON. La validación combina coherencia de configuración con ensayos de dependencias sanas y caídas, cancelación y ausencia de secretos.

```
Aplicación al proyecto del curso:

Endpoint liveness  → /health/live.
Endpoint readiness → /health/ready.
Comprobaciones     → SQL y broker cuando se habilitan workers.
Formato            → JSON.
Validación         → coherencia y dependencias reales.
```

- **Línea 3:** `Endpoint liveness → /health/live.` → /health/live.
- **Línea 4:** `Endpoint readiness → /health/ready.` → /health/ready.
- **Línea 5:** `Comprobaciones → SQL y broker cuando se habilitan workers.` → SQL y broker cuando se habilitan workers.
- **Línea 6:** `Formato → JSON.` → JSON.
- **Línea 7:** `Validación → coherencia y dependencias reales.` → coherencia y dependencias reales.

La aplicación de health checks prepara el proyecto para los puntos 7.3 y 7.4. El punto 7.3 estudiará la gestión de excepciones. El punto 7.4 estudiará el retry y el circuit breaker mediante Polly. El punto 7.2 establece la base que se aplicará en los dos puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El módulo añade dos rutas con significado diferente y conserva /health de Inventario. MapSaludAceria expone live sin checks externos y ready con tres comprobaciones SQL en Producción y una en Inventario; RabbitMQ se añade si WorkersActivos está habilitado. El contrato se verifica sobre las API ejecutadas, además del catálogo.

Ejemplo razonado. La secuencia de ensayo es estado inicial sano, dependencia detenida, observación de live/ready y recuperación. Repetirla para SQL y RabbitMQ muestra que el estado depende de recursos efectivos, no de una variable booleana manipulada en memoria.

Riesgo, alternativa y límite. Readiness no acredita que todos los mensajes estén procesados ni que la proyección ya incluya la última orden. Mantén esa distinción al combinar salud con el recorrido de M6. Tampoco declares cobertura de SQL real a partir de un doble, SQLite o un proveedor InMemory.

Comprobación y aprendizaje. Anota por escenario los estados esperados y observados, los workers habilitados y el recurso afectado. Una prueba debe fallar si la dependencia requerida no está disponible al inicio, en lugar de omitirse. Así la evidencia describe un ensayo realizado y no una intención.

Resumen rápido de la teoría

- Un health check expone el estado del servicio.
- Los tipos de health check son liveness, readiness y startup.
- Los health checks aportan señales de salud; la disponibilidad funcional requiere medir operaciones definidas.
- ASP.NET Core ofrece soporte nativo para health checks.
- Los health checks personalizados comprueban recursos específicos.
- La respuesta se puede formatear en JSON.
- Los contenedores usan health checks para detectar servicios no disponibles.
- Los balanceadores usan health checks para decidir qué instancias reciben tráfico.
- Las buenas prácticas son endpoints separados, solo recursos críticos, startup check, sin información sensible y tiempos configurados.

Punto 7.3 — Gestión de excepciones

Módulo 7: Observabilidad y resiliencia

Objetivos de aprendizaje:

- Comprender la diferencia entre excepciones esperadas y excepcionales.
- Identificar el papel del manejador global de excepciones.
- Comprender el formato problem details como respuesta uniforme de error.
- Comprender la relación entre excepciones y el patrón Result.
- Aplicar los conceptos al comportamiento real de AceriaApp y contrastar sus resultados con pruebas.

Desarrollo teórico

Bloque 1 — Definición de gestión de excepciones

La gestión de excepciones es el conjunto de mecanismos que permiten controlar los errores que ocurren durante la ejecución del sistema. La gestión incluye la captura, el registro, la traducción y la respuesta al cliente. La gestión se aplica de forma uniforme en todos los servicios. La gestión evita que las excepciones no controladas lleguen al cliente sin información. La gestión permite que el cliente reciba una respuesta coherente y que el equipo pueda diagnosticar el problema.

```
Gestión de excepciones:

Excepción → captura → registro → traducción → respuesta.
```

- **Línea 3:** `Excepción → captura → registro → traducción → respuesta.` → flujo de la gestión.

La gestión de excepciones se caracteriza por tres propiedades. La primera es que captura las excepciones no controladas. La segunda es que registra la excepción con contexto. La tercera es que traduce la excepción en una respuesta uniforme. Las tres propiedades permiten que el sistema sea predecible y diagnosticable.

La aplicación de la gestión de excepciones tiene dos consecuencias prácticas. La primera es que el cliente recibe una respuesta coherente. La segunda es que el equipo dispone de información para diagnosticar. La combinación de ambas permite que el sistema sea robusto y observable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Gestionar una excepción implica decidir qué parte se maneja y qué información sale del proceso. El límite HTTP traduce un fallo técnico a un contrato público, pero no deshace automáticamente una transacción ya confirmada ni recupera un proceso que terminó de forma abrupta.

Ejemplo razonado. Si un endpoint falla antes de escribir la respuesta, el manejador puede producir un 500 uniforme y añadir correlación. Si ya envió cabeceras o bytes, no puede sustituirlos por otro documento JSON válido. Ese límite forma parte del diseño y de sus pruebas.

Riesgo, alternativa y límite. Un catch general que devuelve éxito oculta el fallo y permite que el cliente tome decisiones con información falsa. El manejador debe conservar la condición de error. Cuando el cliente canceló, responder como si hubiera un fallo interno añade ruido y puede escribir en una conexión ya cerrada.

Comprobación y aprendizaje. Provoca un error antes de iniciar la respuesta y otro después de iniciarla. Verifica un único documento de error en el primer caso y ausencia de reescritura en el segundo. La diferencia explica por qué «manejador global» no significa que todo fallo pueda repararse.

Bloque 2 — Excepciones esperadas y excepcionales

Los errores se distinguen entre resultados esperados y fallos inesperados; no todos necesitan representarse con excepciones. Las excepciones esperadas son parte del flujo normal del negocio: un recurso que no existe, una validación que falla, un conflicto de estado. Los fallos inesperados interrumpen el flujo normal: un fallo de disco o una violación de contrato. Los fallos de red deben preverse operativamente, aunque una biblioteca los represente mediante excepciones. Las excepciones esperadas se representan con el patrón Result. Las excepciones excepcionales se representan con excepciones. La distinción es la base del uso correcto de la gestión.

```
Excepciones esperadas    → patrón Result.
Excepciones excepcionales → excepciones.
```

- **Línea 1:** `Excepciones esperadas → patrón Result.` → se representan como valores.
- **Línea 2:** `Excepciones excepcionales → excepciones.` → se representan como excepciones.

La distinción tiene dos consecuencias prácticas. La primera es que las excepciones esperadas no se capturan con `try/catch`. La segunda es que las excepciones excepcionales se capturan en el manejador global. La combinación de ambas permite que el flujo de errores sea explícito y controlado.

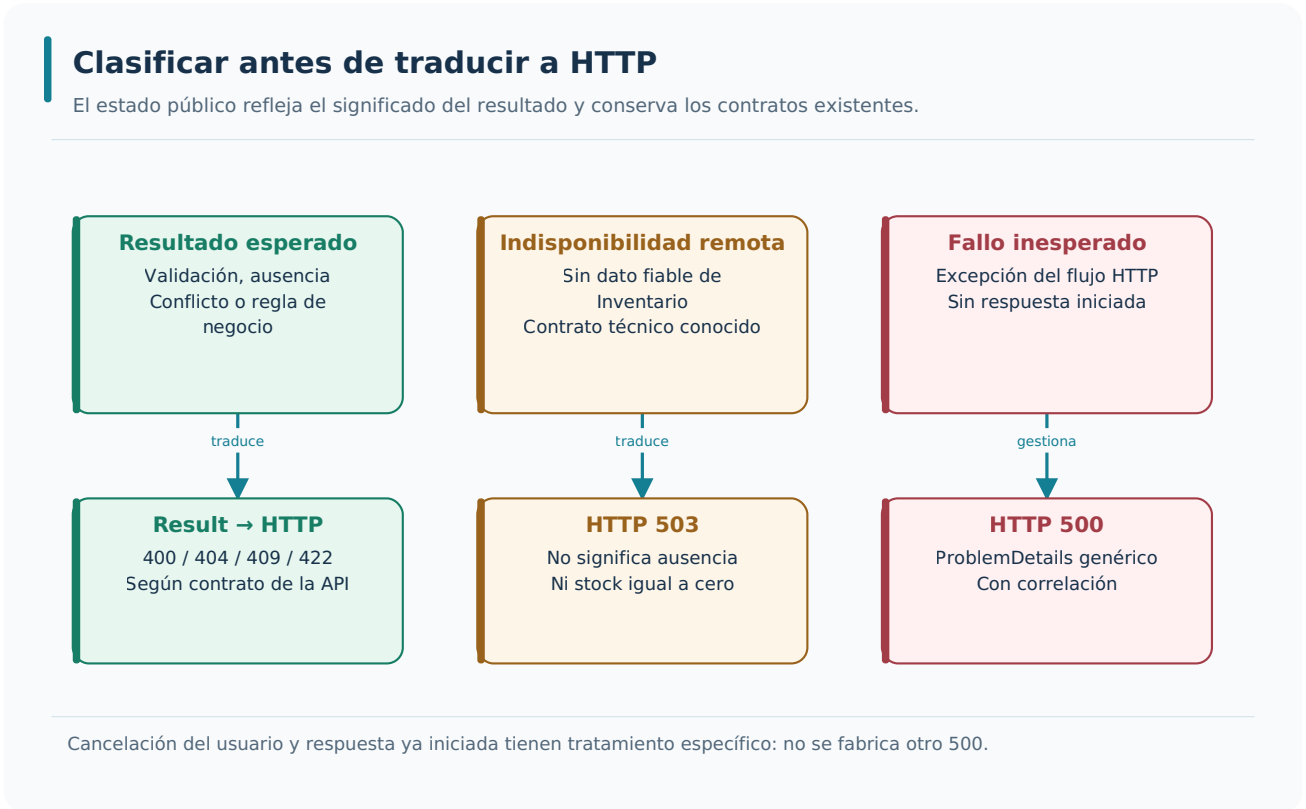
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Es más preciso hablar de errores esperados y fallos inesperados que de «excepciones esperadas». Un resultado de negocio puede ser negativo sin que el programa funcione mal. El contrato del caso de uso expresa ese resultado; una excepción interrumpe una ruta que no pudo completarse normalmente.

Ejemplo razonado. Una orden inexistente produce el resultado previsto y el endpoint responde 404. Un conflicto de estado conserva 409. La indisponibilidad técnica de Inventario se distingue de la ausencia de material y mantiene 503. No conviene convertir las tres situaciones en el mismo 500.

Riesgo, alternativa y límite. Un fallo de red es esperable en sentido operativo, aunque una biblioteca lo represente con excepción. La clasificación depende del contrato de la capa. En infraestructura se traduce a una indisponibilidad conocida; el dominio no necesita conocer HttpRequestException.

Comprobación y aprendizaje. Para cada caso escribe: condición, representación en aplicación y respuesta HTTP. Incluye validación, ausencia, conflicto, indisponibilidad y error de programación. Una tabla que solo enumera tipos de Exception omite los resultados ordinarios y no explica el contrato completo.



Interpretación del gráfico: Los resultados esperados conservan su traducción HTTP; los fallos inesperados se gestionan en el límite. Cancelación y respuesta iniciada requieren caminos específicos.

Bloque 3 — Excepciones personalizadas

Las excepciones personalizadas representan condiciones específicas del dominio. Cada excepción personalizada hereda de `Exception` o de una clase base común. La excepción incluye un mensaje descriptivo y, opcionalmente, un código de error. Las excepciones personalizadas permiten que el manejador global las traduzca en respuestas específicas. Una excepción específica puede lanzarse desde la capa que detecta la condición. La ausencia ordinaria de una orden en AceriaApp sigue representándose mediante Result; el ejemplo ilustra sintaxis, no sustituye ese contrato.

```
public class OrdenNoEncontradaException : Exception
{
    public Guid OrdenId { get; }

    public OrdenNoEncontradaException(Guid ordenId)
        : base($"La orden con identificador '{ordenId}' no existe.")
    {
        OrdenId = ordenId;
    }
}
```

- **Línea 1:** `public class OrdenNoEncontradaException : Exception` → excepción personalizada.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `public Guid OrdenId { get; }` → propiedad con el identificador.
- **Línea 5:** `public OrdenNoEncontradaException(Guid ordenId)` → constructor.
- **Línea 6:** `: base($"La orden con identificador '{ordenId}' no existe.")` → invoca al constructor base.
- **Línea 7:** `{` → abre el cuerpo del constructor.
- **Línea 8:** `OrdenId = ordenId;` → asigna el identificador.
- **Línea 9:** `}` → cierra el constructor.
- **Línea 10:** `}` → cierra la clase.

La excepción personalizada se caracteriza por tres propiedades. La primera es que representa una condición específica del dominio. La segunda es que incluye información contextual. La tercera es que se puede traducir en una respuesta específica. Las tres propiedades permiten que el manejador global produzca respuestas coherentes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Una excepción personalizada aporta un significado técnico que la capa consumidora puede reconocer. No es una obligación convertir cada resultado de negocio en una clase de excepción. OrdenNoEncontradaException sirve para entender sintaxis o integrar código heredado; AceriaApp mantiene la ausencia ordinaria como Result.

Ejemplo razonado. InventarioNoDisponibleException comunica que no se obtuvo un dato remoto fiable. El adaptador la usa después de agotar su política o recibir un fallo de transporte. El consumidor sabe distinguir esa situación del resultado «material inexistente» sin depender de nombres de excepciones de Polly.

Riesgo, alternativa y límite. Un mensaje descriptivo interno no debe convertirse directamente en detail de ProblemDetails. Tampoco se debe usar texto de Message para escoger un estado HTTP: cambia con la traducción, el proveedor o la versión. Usa tipos o códigos estables y un mensaje público controlado.

Comprobación y aprendizaje. Cambia el mensaje de una excepción de prueba sin cambiar su tipo. El estado HTTP debe permanecer igual y el mensaje libre debe seguir oculto. Después verifica que un Result de ausencia mantiene 404 sin recorrer el manejador de fallos inesperados.

Bloque 4 — Manejador global de excepciones

El manejador global de excepciones captura excepciones no controladas del pipeline HTTP situado después de él, cuando todavía es posible gestionarlas. El manejador se registra como middleware en el arranque. El manejador captura la excepción, registra el evento y devuelve una respuesta al cliente. La respuesta sigue el formato problem details. La respuesta incluye estado y título públicos; detalle e

instancia son opcionales y no deben exponer información interna. El manejador evita que las excepciones se propaguen al cliente sin información.

```
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<ManejadorExcepciones>();

var app = builder.Build();
app.UseMiddleware<CorrelacionMiddleware>();
app.UseExceptionHandler();
app.MapControllers();
app.Run();
```

- **Línea 1:** `builder.Services.AddProblemDetails();` → registra los servicios del formato de error.
- **Línea 2:** `builder.Services.AddExceptionHandler<ManejadorExcepciones>();` → registra la implementación IExceptionHandler completa de las prácticas.
- **Línea 4:** `var app = builder.Build();` → construye el host después de registrar servicios.
- **Línea 5:** `app.UseMiddleware<CorrelacionMiddleware>();` → establece correlación antes de gestionar errores.
- **Línea 6:** `app.UseExceptionHandler();` → activa el middleware que usa el manejador.
- **Línea 7:** `app.MapControllers();` → mapea controladores detrás del límite de errores.
- **Línea 8:** `app.Run();` → ejecuta la aplicación; es un fragmento de integración del arranque.

El manejador global se caracteriza por tres propiedades. La primera es que centraliza el manejo de fallos del flujo HTTP que puede interceptar. La segunda es que registra la excepción. La tercera es que devuelve una respuesta uniforme. Las tres propiedades permiten que el sistema sea predecible.

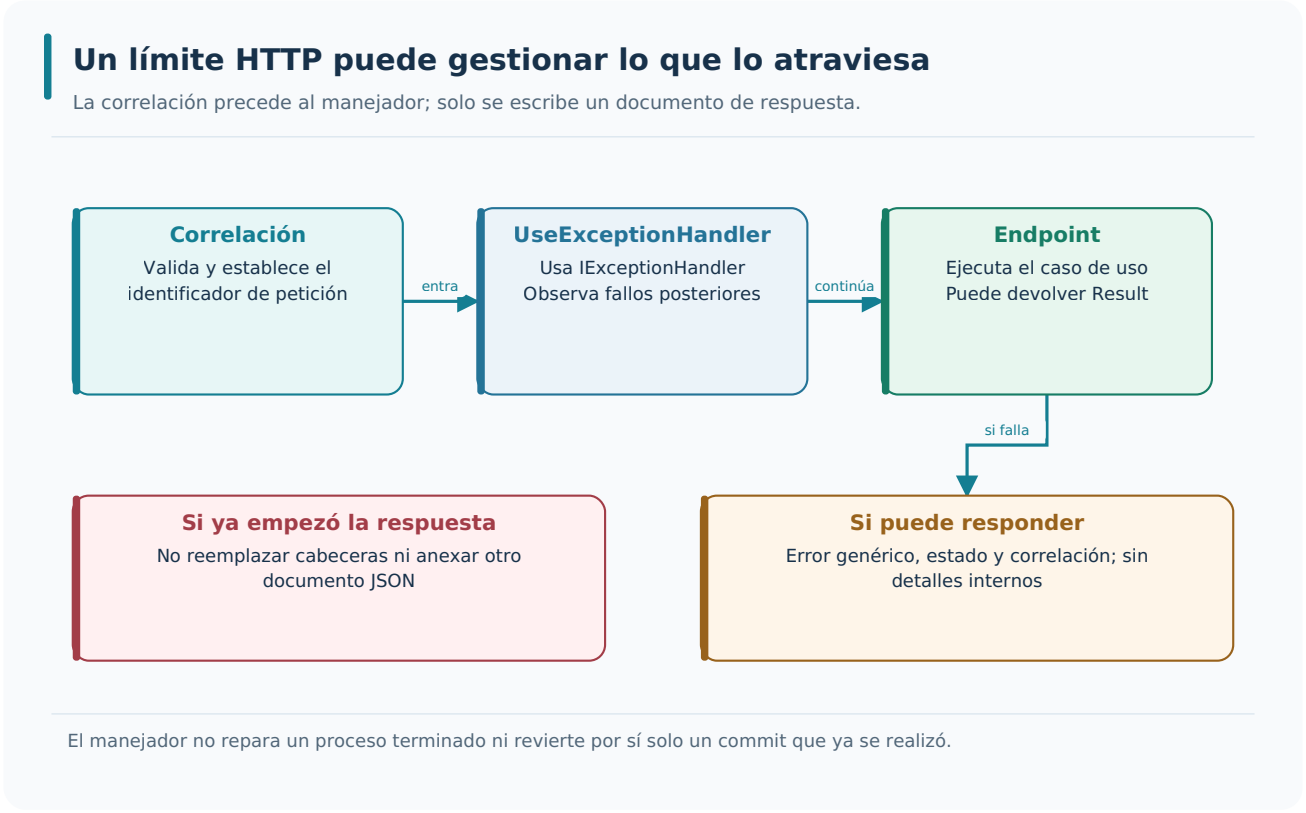
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. ASP.NET Core 8 ofrece IExceptionHandler para separar el comportamiento del manejador del arranque. AddExceptionHandler registra la implementación; AddProblemDetails registra el servicio de escritura; UseExceptionHandler coloca el middleware. Los tres cumplen papeles distintos y se necesitan en el orden apropiado.

Ejemplo razonado. La implementación comprueba cancelación y respuesta iniciada, registra contexto permitido y genera un ProblemDetails 500 con título genérico, traceId y correlationId. Si RequestAborted está cancelado antes de iniciar la respuesta, registra estado 499 sin cuerpo; es una convención local, no un estado HTTP estándar ni una garantía de entrega al cliente desconectado.

Riesgo, alternativa y límite. TryHandleAsync devuelve true solo si se ha gestionado la excepción. Devolver false no significa fabricar una respuesta de éxito; permite continuar el mecanismo de manejo. Si el escritor no acepta el formato solicitado, la implementación completa debe disponer de una salida coherente sin duplicar la respuesta.

Comprobación y aprendizaje. Prueba el contrato completo: status HTTP, Content-Type, campos públicos y correlación. Envía también un Accept que no seleccione JSON y verifica el comportamiento de fallback documentado. No basta con llamar al método aislado si el host nunca registra UseExceptionHandler.



Interpretación del gráfico: La correlación debe existir antes del manejo. Una respuesta ya iniciada no puede reemplazarse por otro documento de error.

Bloque 5 — Problem details como formato de error

El formato problem details está definido actualmente en RFC 9457, que sustituye RFC 7807, y describe la estructura de un error en una API REST. Los campos principales son `type`, `title`, `status`, `detail` e `instance`. El campo `type` identifica el tipo de error con una URI. El campo `title` es un resumen legible. El campo `status` es el código HTTP. El campo `detail` explica el error concreto. El campo `instance` identifica la instancia del error. El formato permite uniformar errores; este módulo lo aplica a fallos inesperados y conserva los contratos de errores esperados heredados.

```
{
  "type": "about:blank",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "La orden con identificador '123' no existe.",
  "instance": "/api/v1/ordenes/123"
}
```

- **Línea 1:** { → abre el objeto.
- **Línea 2:** `"type": "about:blank",` → tipo del error.
- **Línea 3:** `"title": "Recurso no encontrado",` → título.
- **Línea 4:** `"status": 404,` → código.
- **Línea 5:** `"detail": "La orden con identificador '123' no existe.",` → detalle.
- **Línea 6:** `"instance": "/api/v1/ordenes/123"` → instancia.
- **Línea 7:** } → cierra el objeto.

El formato problem details se caracteriza por tres propiedades. La primera es que estructura la respuesta de error. La segunda es que es uniforme en toda la API. La tercera es que facilita el consumo por parte de los clientes. Las tres propiedades permiten que las respuestas de error sean predecibles.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. ProblemDetails es un formato de representación, no un catálogo universal de errores. RFC 9457 sustituye RFC 7807. Type identifica la clase del problema; title resume; status debe concordar con el estado HTTP; detail e instance pueden omitirse si no aportan información pública segura.

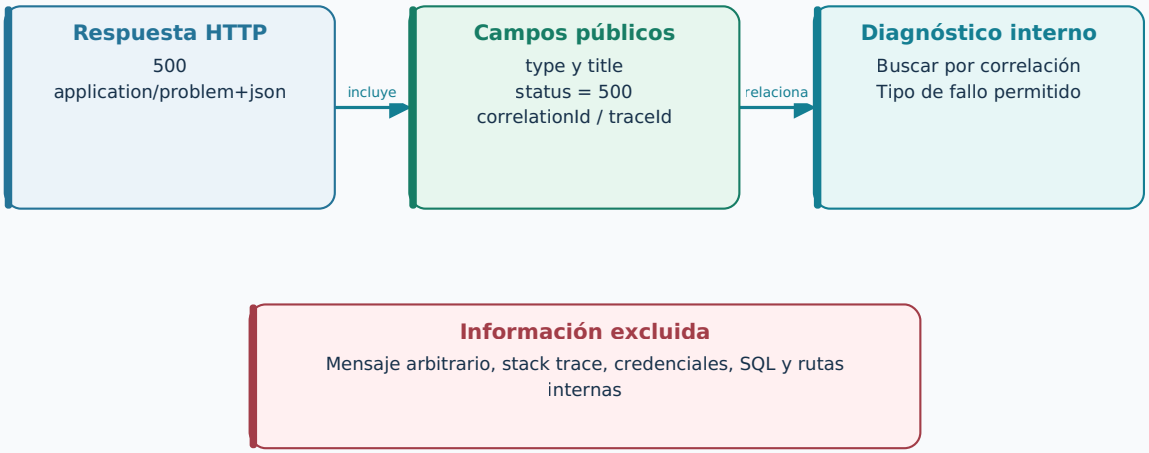
Ejemplo razonado. Un 500 del laboratorio usa «Error interno» y un identificador de correlación. El consumidor puede mostrar un mensaje genérico y guardar ese identificador para soporte. No necesita rutas de archivos, SQL ni una pila de llamadas para interpretar que la operación falló.

Riesgo, alternativa y límite. No uses una URL ficticia como si fuera documentación de errores publicada. About:blank es válido cuando se usa la semántica del estado HTTP. La forma de los errores esperados heredados no cambia automáticamente por añadir AddProblemDetails; el contrato existente debe conservarse deliberadamente.

Comprobación y aprendizaje. Deserializa la respuesta como JSON y verifica tipos y valores. Contrasta status del cuerpo con el código de transporte. Comprueba que el centinela secreto de la excepción no está presente y que las extensiones traceId o correlationId contienen únicamente identificadores previstos.

El error público y el diagnóstico tienen límites

ProblemDetails sigue RFC 9457 y debe concordar con el estado HTTP del transporte.



detail e instance son opcionales. Un título público genérico no impide localizar el incidente por su identidad.

Interpretación del gráfico: El contrato público describe el problema y su identidad de diagnóstico; omite detalles internos y concuerda con el estado HTTP.

Bloque 6 — Traducción de excepciones a códigos HTTP

Cuando un contrato utiliza excepciones conocidas, su traducción a HTTP puede centralizarse en el manejador. En AceriaApp, los errores esperados siguen traduciéndose desde Result y la indisponibilidad remota mantiene 503. Cada tipo de excepción se traduce a un código específico. La

excepción `OrdenNoEncontradaException` se traduce a 404. La excepción `ValidacionException` se traduce a 400. La excepción `ConflictoException` se traduce a 409. La excepción `RecursoNoProcesableException` se traduce a 422. Las excepciones no reconocidas se traducen a 500. La traducción se realiza con un `switch` sobre el tipo de excepción.

```
var (status, title) = exception switch
{
    OrdenNoEncontradaException => (StatusCodes.Status404NotFound, "Recurso no encontrado"),
    ValidacionException => (StatusCodes.Status400BadRequest, "Error de validación"),
    ConflictoException => (StatusCodes.Status409Conflict, "Conflicto de estado"),
    RecursoNoProcesableException => (StatusCodes.Status422UnprocessableEntity, "No procesable"),
    _ => (StatusCodes.Status500InternalServerError, "Error interno")
};
```

- **Línea 1:** `var (status, title) = exception switch` → comienza la traducción.
- **Línea 2:** `{` → abre el bloque.
- **Línea 3:** `OrdenNoEncontradaException => (StatusCodes.Status404NotFound, "Recurso no encontrado"),` → caso 404.
- **Línea 4:** `ValidacionException => (StatusCodes.Status400BadRequest, "Error de validación"),` → caso 400.
- **Línea 5:** `ConflictoException => (StatusCodes.Status409Conflict, "Conflicto de estado"),` → caso 409.
- **Línea 6:** `RecursoNoProcesableException => (StatusCodes.Status422UnprocessableEntity, "No procesable"),` → caso 422.
- **Línea 7:** `_ => (StatusCodes.Status500InternalServerError, "Error interno")` → caso por defecto.
- **Línea 8:** `};` → cierra el bloque.

La traducción se caracteriza por tres propiedades. La primera es que cada excepción tiene un código específico. La segunda es que las excepciones no reconocidas tienen un código por defecto. La tercera es que la traducción es centralizada. Las tres propiedades permiten que las respuestas de error sean coherentes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La traducción pertenece al borde de la aplicación, porque HTTP es un detalle del adaptador de entrada. Los errores esperados mantienen 400, 404, 409 o 422 según el contrato ya definido. Los fallos inesperados reciben 500 y la indisponibilidad remota conocida mantiene 503.

Ejemplo razonado. Una validación de formato puede justificar 400; una regla de negocio que impide procesar una petición válida puede justificar 422. No se decide buscando una palabra en el mensaje. En una API consistente, el mismo código de aplicación se traduce del mismo modo en todos los endpoints.

Riesgo, alternativa y límite. Capturar ArgumentException globalmente y devolver 400 puede ocultar un bug interno que lanza ese tipo. Solo traduce tipos de excepción cuyo contrato y origen conoces. El ejemplo de switch es un adaptador didáctico para clases explícitas, no una razón para abandonar Result.

Comprobación y aprendizaje. Ejecuta una matriz con un caso real de cada estado preservado y un fallo desconocido. Comprueba que el manejador nuevo no intercepta resultados ya resueltos. Incluye indisponibilidad de Inventario para evitar que una mejora de formato cambie accidentalmente 503 por 500.

Bloque 7 — Excepciones y logging

Las excepciones se registran en el manejador global con el nivel adecuado. Las excepciones esperadas se registran con nivel Information o Warning. Las excepciones excepcionales se registran con nivel Error o Critical. El registro incluye contexto permitido y tipo de excepción; este curso evita publicar mensajes, pilas y valores libres que puedan contener datos sensibles. El registro de excepciones es la base del diagnóstico. El registro se realiza antes de devolver la respuesta al cliente.

```
Log.Error("Fallo no controlado de tipo {ExceptionType}", exception.GetType().Name);
```

- **Línea 1:** `Log.Error("Fallo no controlado de tipo {ExceptionType}", exception.GetType().Name);` → registra el tipo con una plantilla estable; LogContext aporta correlación sin copiar la excepción completa.

El registro de excepciones se caracteriza por tres propiedades. La primera es que el evento identifica el tipo de fallo y permite correlacionarlo. La segunda es que se añaden propiedades del contexto. La tercera es que el nivel se ajusta al tipo de excepción. Las tres propiedades permiten que el diagnóstico sea preciso.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El registro final debe tener un único responsable. El resumen HTTP responde cuánto duró y cómo terminó; el evento de excepción añade el tipo de fallo y su correlación. Ambos pueden coexistir porque responden preguntas diferentes, siempre que no se replique la excepción en cada capa.

Ejemplo razonado. Ante InvalidOperationException, Serilog registra un evento de error con nombre de tipo y contexto validado. El resumen muestra 500. El cliente devuelve la correlación al equipo y este localiza ambos eventos sin tener que buscar el texto de un mensaje interno.

Riesgo, alternativa y límite. La gravedad depende del impacto: un fallo aislado no exige Critical. El middleware y Polly pueden emitir excepciones completas, por lo que el laboratorio limita esas categorías y genera eventos propios seguros. Esta decisión no modifica ni audita todos los logs heredados de los workers de M6 o de EF.

Comprobación y aprendizaje. Provoca un error con una query y una excepción que contengan centinelas distintos. Verifica que ninguno aparece en los nuevos eventos propios de error y en el resumen HTTP; comprueba también que no heredan RequestPath. Cuenta los eventos esperados y relaciona sus identificadores para comprobar diagnóstico y minimización de datos.

Bloque 8 — Excepciones y patrón Result

El patrón Result y las excepciones se complementan. Las excepciones esperadas se representan con `Result.Failure`. Las excepciones excepcionales se representan con excepciones. La combinación permite que el flujo de errores esperados sea explícito y que los errores excepcionales se capturen en el manejador global. La separación entre ambos es la base del uso correcto de la gestión.

```
public async Task<Result<Guid>> Handle(CrearOrdenCommand request, CancellationToken ct)
{
    var cliente = await _clienteRepositorio.GetByIdAsync(request.ClienteId, ct);
    if (cliente is null)
    {
        return Result<Guid>.Failure("El cliente no existe.");
    }

    var orden = new Orden(Guid.NewGuid(), cliente, request.Cantidad);
    await _ordenRepositorio.AddAsync(orden, ct);
    await _unitOfWork.CommitAsync(ct);
    return Result<Guid>.Success(orden.Id);
}
```

- **Línea 1:** `public async Task<Result<Guid>> Handle(CrearOrdenCommand request, CancellationToken ct)` → método de manejo.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `var cliente = await _clienteRepositorio.GetByIdAsync(request.ClienteId, ct);` → recupera el cliente.
- **Línea 4:** `if (cliente is null)` → valida que existe.
- **Línea 5:** `{` → abre el bloque.
- **Línea 6:** `return Result<Guid>.Failure("El cliente no existe.");` → devuelve un fallo esperado.
- **Línea 7:** `}` → cierra el bloque.
- **Línea 9:** `var orden = new Orden(Guid.NewGuid(), cliente, request.Cantidad);` → crea la orden.
- **Línea 10:** `await _ordenRepositorio.AddAsync(orden, ct);` → persiste.
- **Línea 11:** `await _unitOfWork.CommitAsync(ct);` → confirma la transacción.
- **Línea 12:** `return Result<Guid>.Success(orden.Id);` → devuelve el éxito.
- **Línea 13:** `}` → cierra el método.

La combinación se caracteriza por tres propiedades. La primera es que los errores esperados no lanzan excepciones. La segunda es que los errores excepcionales se capturan en el manejador global. La tercera es que el flujo de errores es explícito. Las tres propiedades permiten que el sistema sea coherente.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Result hace explícita la posibilidad de un resultado de negocio negativo en la firma y obliga al consumidor a considerarlo. Las excepciones conservan su papel para fallos técnicos y condiciones no previstas. El ejemplo es conceptual: las prácticas usan los contratos Result y entidades que el proyecto ya contiene.

Ejemplo razonado. Un cliente inexistente devuelve Failure antes de construir y guardar una orden. Si el commit de SQL falla, no se devuelve Success. El límite HTTP transforma ese fallo según su contrato, sin declarar que se haya persistido una operación que no llegó a confirmar.

Riesgo, alternativa y límite. Atrapar cualquier excepción y convertirla en Failure("no existe") confunde error de infraestructura con ausencia. También hay que respetar cancelación: no debe transformarse en una validación fallida. El token se propaga a consultas y commit para que el trabajo pueda detenerse.

Comprobación y aprendizaje. Comprueba tres rutas distintas: ausencia conocida, persistencia correcta y fallo del repositorio. Observa valor devuelto, escrituras realizadas y respuesta HTTP. La separación demuestra que un sistema puede manejar errores de negocio de forma normal sin ocultar los fallos técnicos.

Bloque 9 — Buenas prácticas de gestión de excepciones

Las buenas prácticas de gestión de excepciones son cinco. La primera es no usar excepciones para el flujo normal. La segunda es capturar solo las excepciones que se pueden manejar. La tercera es no capturar excepciones genéricas sin propósito. La cuarta es registrar las excepciones con contexto. La quinta es devolver respuestas uniformes al cliente. La aplicación de las cinco prácticas produce un sistema robusto y diagnosticable.

```
Buenas prácticas de gestión de excepciones:

1. No usar excepciones para el flujo normal.
2. Capturar solo las excepciones manejables.
3. No capturar excepciones genéricas sin propósito.
4. Registrar las excepciones con contexto.
5. Devolver respuestas uniformes.
```

- **Línea 3:** `1. No usar excepciones para el flujo normal.` → primera práctica.
- **Línea 4:** `2. Capturar solo las excepciones manejables.` → segunda práctica.
- **Línea 5:** `3. No capturar excepciones genéricas sin propósito.` → tercera práctica.
- **Línea 6:** `4. Registrar las excepciones con contexto.` → cuarta práctica.
- **Línea 7:** `5. Devolver respuestas uniformes.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen un flujo de errores explícito. La segunda es que facilitan el diagnóstico. La tercera es que mantienen la coherencia de la API. Las tres propiedades permiten que la gestión de excepciones sea un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Captura cuando puedes recuperar, traducir con contexto o liberar recursos; deja que el resto llegue al límite apropiado. La centralización reduce duplicación, pero no sustituye las transacciones, la disposición de recursos ni el tratamiento específico de cancelación.

Ejemplo razonado. Un endpoint de diagnóstico genera un fallo controlado para verificar el manejador. Solo se registra cuando el entorno es Development y una opción explícita lo habilita. En Production la ruta no existe, aunque la opción esté activada por error.

Riesgo, alternativa y límite. Una bandera de configuración sin comprobación de entorno deja expuesta una función de fallo intencional. Por otro lado, la Developer Exception Page revela detalles útiles para desarrollo y no debe convertirse en el contrato público del servicio. Los ensayos deben distinguir ambos comportamientos.

Comprobación y aprendizaje. Prueba la combinación de entorno y opción: Development activado permite el ensayo, Development desactivado y Production no exponen la ruta. Comprueba además cancelación y respuesta iniciada. Son casos donde un manejador aparentemente correcto puede producir un segundo error.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica la gestión de excepciones en el módulo 7. El manejador global captura las excepciones no controladas. Las excepciones esperadas se representan con el patrón Result. Las excepciones excepcionales se traducen en respuestas problem details. El registro de excepciones se realiza con Serilog. La validación mantiene el catálogo y comprueba respuestas HTTP, privacidad, cancelación, respuesta iniciada y ausencia de la ruta de diagnóstico en Production.

```
Aplicación al proyecto del curso:

Manejador global     → IExceptionHandler y UseExceptionHandler.
Formato              → Problem details.
Excepciones esperadas → Result.
Excepciones excepcionales → manejador global.
Registro             → Serilog.
Validación           → coherencia y respuestas reales.
```

- **Línea 3:** `Manejador global → IExceptionHandler y UseExceptionHandler.` → IExceptionHandler y UseExceptionHandler.
- **Línea 4:** `Formato → Problem details.` → Problem details.
- **Línea 5:** `Excepciones esperadas → Result.` → Result.
- **Línea 6:** `Excepciones excepcionales → manejador global.` → manejador global.
- **Línea 7:** `Registro → Serilog.` → Serilog.
- **Línea 8:** `Validación → coherencia y respuestas reales.` → coherencia y respuestas reales.

La aplicación de la gestión de excepciones prepara el proyecto para el punto 7.4. El punto 7.4 estudiará el retry y el circuit breaker mediante Polly. El punto 7.3 establece la base que se aplicará en el punto 7.4.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El resultado del punto es un límite HTTP predecible: fallos inesperados con ProblemDetails, resultados esperados preservados y contexto seguro para diagnóstico. La biblioteca compartida evita que las API de Producción e Inventario implementen versiones incompatibles de esa regla.

Ejemplo razonado. Una consulta de Inventario puede fallar por transporte, por ausencia o por un error interno del servidor. El cliente conserva esas distinciones. En el punto siguiente, la política de resiliencia decide qué fallos transitorios merecen otro intento antes de devolver indisponibilidad.

Riesgo, alternativa y límite. Centralizar el formato no permite ignorar quién es responsable de la recuperación. El manejador HTTP no debe reejecutar una operación de negocio completa después de un fallo desconocido. Un commit pudo haberse realizado antes de perderse la respuesta.

Comprobación y aprendizaje. Recorre la tabla de estados anterior y explica qué capa toma cada decisión. Verifica que la correlación atraviesa el error igual que una respuesta correcta. Un diagnóstico útil conecta la respuesta pública y los eventos internos sin exponer la implementación al cliente.

Resumen rápido de la teoría

- La gestión de excepciones controla los errores de forma uniforme.
- Las excepciones se clasifican en esperadas y excepcionales.
- Las excepciones esperadas se representan con el patrón Result.
- Las excepciones excepcionales se representan con excepciones.
- Las excepciones personalizadas representan condiciones específicas del dominio.
- El manejador centraliza fallos del pipeline HTTP que puede interceptar; cancelación y respuesta iniciada requieren tratamiento específico.
- El formato problem details estructura las respuestas de error.
- La traducción de fallos conocidos se centraliza sin sustituir los resultados de negocio existentes.
- Las excepciones se registran con el nivel adecuado.

- Las buenas prácticas son: no usar excepciones para el flujo normal, capturar solo lo manejable, no capturar genéricas sin propósito, registrar con contexto y devolver respuestas uniformes.

Punto 7.4 — Retry y Circuit Breaker mediante Polly

Módulo 7: Observabilidad y resiliencia

Objetivos de aprendizaje:

- Comprender los patrones de resiliencia Retry y Circuit Breaker.
- Identificar las situaciones en las que se aplica cada patrón.
- Comprender la combinación de patrones en una política de resiliencia.
- Comprender la integración de Polly con ASP.NET Core.
- Aplicar los conceptos al comportamiento real de AceriaApp y contrastar sus resultados con pruebas.

Desarrollo teórico

Bloque 1 — Definición de resiliencia

La resiliencia es la capacidad de un sistema para recuperarse de fallos y seguir operando. En sistemas distribuidos, los fallos son inevitables: la red puede fallar, los servicios pueden no responder, las dependencias pueden degradarse. La resiliencia se implementa con patrones que permiten al sistema limitar el impacto de los fallos y comunicar la indisponibilidad cuando no pueden recuperarse. Los patrones de resiliencia más usados son el retry, el circuit breaker, el timeout y el bulkhead. La aplicación conjunta de los patrones produce un sistema robusto.

```
Patrones de resiliencia:

Retry          → reintenta operaciones fallidas.
CircuitBreaker → corta el flujo cuando el destino falla.
Timeout        → limita el tiempo de espera.
Bulkhead       → aísla recursos por operación.
```

- **Línea 3:** `Retry → reintenta operaciones fallidas.` → reintenta operaciones fallidas.
- **Línea 4:** `CircuitBreaker → corta el flujo cuando el destino falla.` → corta el flujo cuando el destino falla.
- **Línea 5:** `Timeout → limita el tiempo de espera.` → limita el tiempo de espera.
- **Línea 6:** `Bulkhead → aísla recursos por operación.` → aísla recursos por operación.

La resiliencia se caracteriza por tres propiedades. La primera es que los fallos se contienen dentro de límites definidos. La segunda es que el sistema puede recuperarse automáticamente cuando el recurso vuelve a estar disponible. La tercera es que la degradación es controlada. Las tres propiedades permiten que el sistema sea robusto y predecible.

La aplicación de la resiliencia tiene dos consecuencias prácticas. La primera es que el sistema tolera fallos transitorios. La segunda es que el sistema evita la saturación por fallos en cascada. La combinación de ambas permite que el sistema sea tolerante a fallos.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La resiliencia limita el impacto de fallos y permite recuperarse cuando las condiciones lo permiten. No elimina los fallos ni garantiza siempre una respuesta exitosa. Devolver indisponibilidad de forma rápida y honesta puede ser mejor resultado que mantener recursos ocupados durante un tiempo indefinido.

Ejemplo razonado. Producción necesita consultar Inventario. Un corte breve puede resolverse con un nuevo intento; una caída sostenida exige dejar de insistir; una respuesta demasiado lenta debe agotar su presupuesto. Cada mecanismo actúa sobre una dimensión diferente del mismo problema.

Riesgo, alternativa y límite. Bulkhead y limitación de concurrencia sirven para aislar recursos, pero no se implementan como un nuevo subsistema en este módulo. Añadir patrones sin medir su interacción aumenta complejidad. La política concreta se limita al cliente de consulta, sin alterar mensajes, contratos o entidades del dominio.

Comprobación y aprendizaje. Clasifica un corte de una petición, una caída de treinta segundos y una respuesta de duración ilimitada. Identifica qué estrategia acota cada caso y cuál será la respuesta cuando no haya recuperación. La explicación debe incluir tanto el éxito recuperado como el fallo final.

Bloque 2 — Patrón Retry

El patrón Retry reintenta una operación fallida un número determinado de veces. El patrón se aplica a fallos transitorios: errores de red, timeouts, respuestas 5xx. El patrón no se aplica a fallos permanentes como validación, 400, 401, 403 o 404. Los estados 408 y 429 son excepciones relevantes dentro de 4xx y pueden tratarse como transitorios en una operación segura. El reintento se realiza con un intervalo entre intentos. El intervalo puede ser fijo, exponencial o con jitter. La elección del intervalo depende del tipo de operación y del destino.

```
var retry = new HttpRetryStrategyOptions
{
    MaxRetryAttempts = 2,
    Delay = TimeSpan.FromMilliseconds(200),
    BackoffType = DelayBackoffType.Exponential,
    UseJitter = true
};
retry.DisableForUnsafeHttpMethods();
```

- **Línea 1:** `var retry = new HttpRetryStrategyOptions` → crea opciones HTTP de Polly v8 con su clasificación transitoria.
- **Línea 2:** `{` → abre las opciones.
- **Línea 3:** `MaxRetryAttempts = 2,` → permite dos reintentos, además del intento inicial.
- **Línea 4:** `Delay = TimeSpan.FromMilliseconds(200),` → establece la demora base.
- **Línea 5:** `BackoffType = DelayBackoffType.Exponential,` → aumenta la espera con retroceso exponencial.
- **Línea 6:** `UseJitter = true` → desincroniza las esperas con variación aleatoria.
- **Línea 7:** `};` → termina las opciones.
- **Línea 8:** `retry.DisableForUnsafeHttpMethods();` → deshabilita reintentos para métodos mutantes; el recorrido solo usa GET.

El patrón Retry se caracteriza por tres propiedades. La primera es que reintenta operaciones fallidas. La segunda es que el intervalo puede ser fijo o exponencial. La tercera es que se aplica a fallos transitorios. Las tres propiedades permiten que el sistema recupere errores temporales.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Dos reintentos significan como máximo tres intentos: el inicial y dos adicionales. Se reintentan fallos de transporte, timeout de intento y estados 408, 429 y 5xx. No se reintentan 400, 401, 403, 404 ni cancelación solicitada por el usuario. La decisión combina semántica del método y del fallo.

Ejemplo razonado. Si Inventario devuelve 503 y después 200, la lectura puede recuperarse. Un 404 es un resultado definitivo de esa consulta y vuelve inmediatamente al consumidor. La demora inicial de 200 ms crece exponencialmente y el jitter desincroniza clientes; no promete tiempos exactos de 200 y 400 ms.

Riesgo, alternativa y límite. Un retry multiplica carga cuando el destino ya está degradado. Además, Retry-After puede pedir una espera mayor que la tolerada por el usuario: el timeout total exterior mantiene el límite. No se debe reintentar BrokenCircuitException para insistir contra un circuito abierto.

Comprobación y aprendizaje. Usa un transporte controlado que devuelva 503, 503 y 200, y cuenta tres envíos. Repite con 404 y con cancelación: solo debe ejecutarse lo permitido. Una prueba de configuración que inspecciona MaxRetryAttempts no demuestra que el predicado de fallos sea correcto.

Bloque 3 — Patrón Circuit Breaker

El patrón Circuit Breaker corta el flujo cuando el destino falla de forma repetida. El circuito tiene tres estados: cerrado, abierto y semiabierto. En estado cerrado, las operaciones se ejecutan con normalidad. Cuando se alcanza el mínimo de muestras y la proporción de fallos alcanza el umbral dentro de la ventana, el circuito pasa a estado abierto. En estado abierto, las operaciones fallan inmediatamente sin intentar contactar con el destino. Tras el tiempo de apertura, una nueva llamada puede realizar la prueba de estado semiabierto. En estado semiabierto, se permite una operación de prueba. Si tiene éxito, el circuito vuelve a estado cerrado. Si falla, vuelve a estado abierto.

```
Estados del circuit breaker:

Cerrado     → operaciones normales.
Abierto     → operaciones fallan inmediatamente.
Semiabierto → operación de prueba.
```

- **Línea 3:** `Cerrado → operaciones normales.` → operaciones normales.
- **Línea 4:** `Abierto → operaciones fallan inmediatamente.` → operaciones fallan inmediatamente.
- **Línea 5:** `Semiabierto → operación de prueba.` → operación de prueba.

El patrón Circuit Breaker se caracteriza por tres propiedades. La primera es que evita la saturación del destino. La segunda es que falla rápido cuando el destino no está disponible. La tercera es que se recupera de forma automática. Las tres propiedades permiten que el sistema sea tolerante a fallos sostenidos.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El circuito de este módulo evalúa una proporción de fallos dentro de una ventana, con un mínimo de muestras. Con cuatro muestras mínimas y proporción 0,5, necesita suficiente actividad antes de abrir. Los fallos no tienen que ser consecutivos. Abierto rechaza nuevas ejecuciones sin llamar al transporte.

Ejemplo razonado. Una secuencia de cuatro resultados, 200, 503, 200, 503, alcanza dos fallos de cuatro dentro de diez segundos. Al abrir, la siguiente llamada falla rápido. Tras cinco segundos, una nueva solicitud puede actuar como prueba de recuperación; esperar no genera por sí mismo tráfico de prueba.

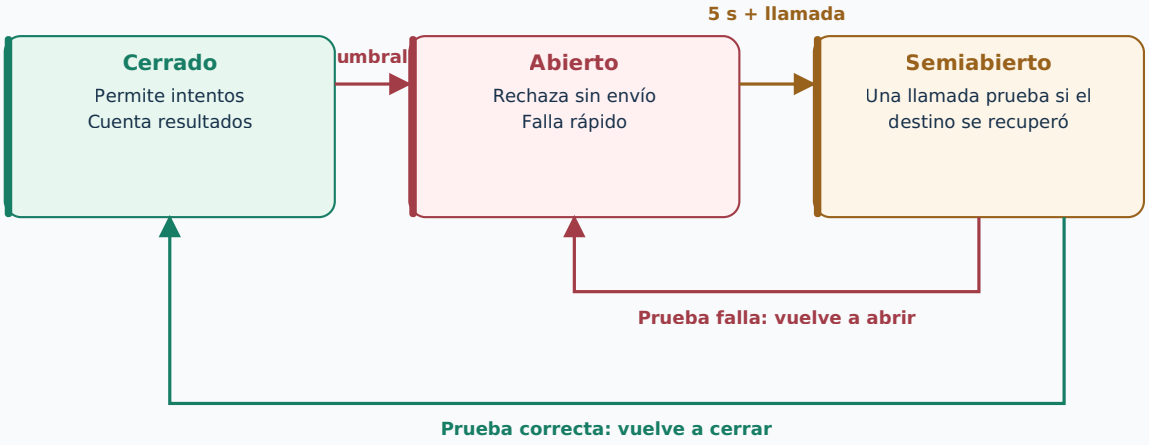
Riesgo, alternativa y límite. El circuito no es un retry y no repara Inventario. Se comparte por cliente y destino; recrearlo en cada llamada borra su historial. Con retry por fuera del circuito, cada intento que llega al transporte contribuye a las muestras, no solamente el resultado final de la petición del usuario.

Comprobación y aprendizaje. Observa contador de envíos y transiciones de estado usando reloj controlado o señales. Verifica apertura, bloqueo sin envío, prueba tras el intervalo y cierre después del éxito. Si la prueba falla, el circuito debe volver a abrir, conservando el periodo de protección.

El circuito conserva estado entre peticiones

Ventana: 10 s · Mínimo: 4 muestras · Ratio de fallo: 0,5 · Apertura: 5 s

Ejemplo de apertura: 200, 503, 200, 503 dentro de la ventana



Esperar cinco segundos no envía una sonda por sí solo; hace falta una nueva llamada.

Interpretación del gráfico: El circuito abre al cumplir muestras y proporción de fallo; una llamada posterior al periodo de apertura prueba recuperación. Una prueba fallida vuelve a abrir.

Bloque 4 — Combinación de patrones

Los patrones de resiliencia se combinan para formar una política completa. La combinación del módulo incluye timeout total, retry, circuit breaker y timeout de intento. El timeout exterior limita el conjunto de trabajo y esperas; el interior limita cada intento. El retry reintenta las operaciones fallidas. El circuit breaker corta el flujo cuando el destino falla de forma sostenida. El orden de aplicación determina el comportamiento observado. En Polly v8 las estrategias se añaden de fuera hacia dentro: total, retry, circuito e intento. No es una secuencia de fases independientes: cada estrategia envuelve a las siguientes.

```
httpClientBuilder.AddResilienceHandler("inventario", pipeline =>
{
    pipeline.AddTimeout(TimeSpan.FromSeconds(6));
    pipeline.AddRetry(retry);
    pipeline.AddCircuitBreaker(circuito);
    pipeline.AddTimeout(TimeSpan.FromSeconds(1));
}).SelectPipelineByAuthority();
```

- **Línea 1:** `httpClientBuilder.AddResilienceHandler("inventario", pipeline =>` → registra una única canalización nombrada en el cliente.

- **Línea 2:** `{` → abre su configuración.
- **Línea 3:** `pipeline.AddTimeout(TimeSpan.FromSeconds(6));` → envuelve todo con el presupuesto total.
- **Línea 4:** `pipeline.AddRetry(retry);` → usa las opciones de retry ya definidas.
- **Línea 5:** `pipeline.AddCircuitBreaker(circuito);` → usa las opciones de circuito definidas en el bloque 6.
- **Línea 6:** `pipeline.AddTimeout(TimeSpan.FromSeconds(1));` → limita el intento que llega al transporte.
- **Línea 7:** `}).SelectPipelineByAuthority();` → comparte estado por autoridad; las opciones se construyen una vez en el arranque.

La combinación se caracteriza por tres propiedades. La primera es que cada patrón cubre un aspecto distinto. La segunda es que el orden de aplicación determina el comportamiento. La tercera es que la combinación produce un sistema robusto. Las tres propiedades permiten que la resiliencia sea completa.

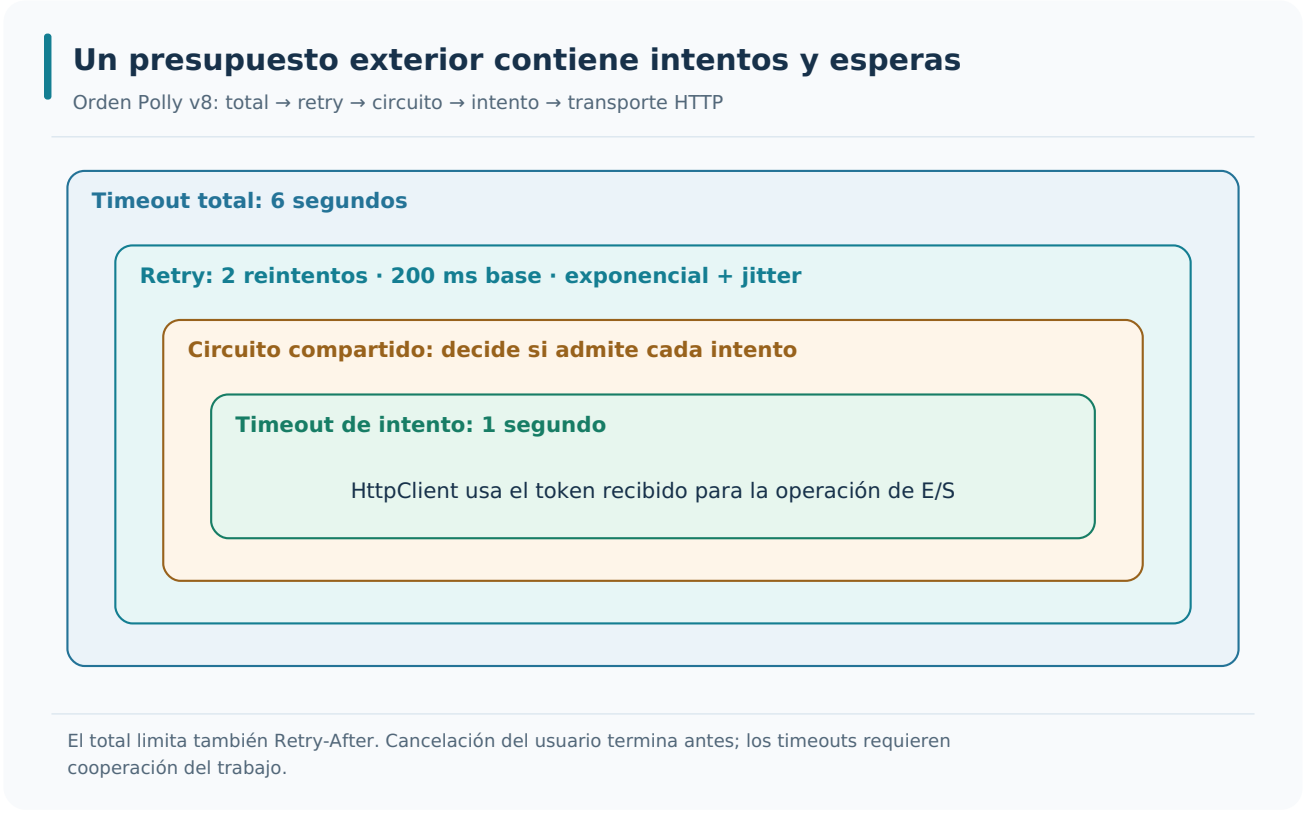
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El orden es de fuera hacia dentro: timeout total de 6 s, retry, circuito, timeout de intento de 1 s y transporte. El límite exterior cubre ejecución y esperas; el interior limita cada intento. El circuito observa resultados de los intentos permitidos, mientras retry decide si habrá otro.

Ejemplo razonado. Una petición puede consumir un segundo en un intento lento, esperar la demora y volver a intentar. Si un Retry-After solicita diez segundos, el presupuesto total corta la operación antes de esperarlos completos. La cancelación del usuario se propaga hacia dentro y puede terminarla antes.

Riesgo, alternativa y límite. Los timeouts de Polly son cooperativos: el callback debe usar el token recibido. Un trabajo que ignora cancelación puede seguir ejecutándose. La práctica lee el cuerpo HTTP dentro del handler para que también quede acotado, y desactiva el timeout propio de HttpClient. Superponer otros handlers completos multiplicaría intentos y límites.

Comprobación y aprendizaje. Dibuja la envoltura y sigue el token desde fuera hasta HttpClient. Ensaya intento lento y espera de retry larga. Comprueba que cada prueba valida el límite correcto: un timeout de intento puede reintentarse, el agotamiento del total concluye la operación completa.



Interpretación del gráfico: Las estrategias se anidan de fuera hacia dentro. El límite total cubre intentos y demoras, y la cancelación se propaga hasta el transporte.

Bloque 5 — Polly como biblioteca de resiliencia

Polly es una biblioteca de .NET para la implementación de patrones de resiliencia. Polly ofrece estrategias de retry, circuit breaker, timeout, fallback y limitación de concurrencia, entre otras. En v8 las estrategias se agregan a un ResiliencePipelineBuilder y la canalización se ejecuta con ExecuteAsync. La integración de HttpClient mediante Microsoft.Extensions.Http.Resilience utiliza AddResilienceHandler. Policy, WrapAsync y AddPolicyHandler pertenecen al estilo anterior y no se mezclan con los ejemplos v8 de este módulo. La biblioteca es la referencia en el ecosistema .NET para la resiliencia.

```
builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>()
    .AddResilienceHandler("inventario", pipeline =>
    {
        pipeline.AddTimeout(TimeSpan.FromSeconds(6));
        pipeline.AddRetry(retry);
        pipeline.AddCircuitBreaker(circuito);
        pipeline.AddTimeout(TimeSpan.FromSeconds(1));
    }).SelectPipelineByAuthority();
```

- **Línea 1:** `builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>()` → registra el cliente tipado que implementa el puerto de consulta.
- **Línea 2:** `.AddResilienceHandler("inventario", pipeline =>` → asocia la canalización al cliente.
- **Línea 3:** `{` → abre las estrategias.
- **Línea 4:** `pipeline.AddTimeout(TimeSpan.FromSeconds(6));` → limita la operación completa.
- **Línea 5:** `pipeline.AddRetry(retry);` → aplica las opciones HTTP de retry previamente configuradas.

- **Línea 6:** `pipeline.AddCircuitBreaker(circuito);` → comparte el circuito asociado al cliente.
- **Línea 7:** `pipeline.AddTimeout(TimeSpan.FromSeconds(1));` → acota cada intento.
- **Línea 8:** `}).SelectPipelineByAuthority();` → separa estado por destino; presupone retry y circuito del bloque 6.

Polly se caracteriza por tres propiedades. La primera es que ofrece políticas para varios patrones. La segunda es que se integra con `HttpClient`. La tercera es que las políticas se combinan. Las tres propiedades permiten que la resiliencia se aplique de forma declarativa.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Polly v8 usa estrategias y ResiliencePipeline. La integración Microsoft.Extensions.Http.Resilience permite configurar una canalización asociada a IHttpClientFactory. Los ejemplos usan AddResilienceHandler, AddRetry y AddCircuitBreaker; las recetas antiguas con Policy y WrapAsync pertenecen a otra API.

Ejemplo razonado. El cliente tipado de Inventario conserva su puerto de aplicación. La resiliencia se añade en infraestructura y en el registro de servicios, sin hacer que el dominio dependa de Polly. La canalización nombrada mantiene estado entre peticiones y SelectPipelineByAuthority separa el circuito por esquema, host y puerto del destino.

Riesgo, alternativa y límite. No mezcles fragmentos v7 y v8 suponiendo que solo cambian los nombres. La configuración, los predicados y la integración difieren. La versión de paquetes se fija en las prácticas y se restaura y compila; la documentación general puede mostrar una versión posterior.

Comprobación y aprendizaje. Localiza en el proyecto el único handler de resiliencia del cliente de Inventario. Verifica que su registro se usa realmente y que dos llamadas sucesivas comparten el circuito. Un pipeline construido dentro de ObtenerAsync podría compilar y, aun así, impedir toda apertura útil.

Bloque 6 — Configuración de políticas

La configuración de políticas se realiza en el arranque de la aplicación. Las políticas se definen con parámetros específicos: número de reintentos, intervalo, umbral de fallos, tiempo de apertura. Los parámetros se ajustan según el tipo de operación y el destino. Los parámetros se documentan en el ADR. La configuración se aplica a todos los clientes que la usan.

```
var retry = new HttpRetryStrategyOptions
{
    MaxRetryAttempts = 2,
    Delay = TimeSpan.FromMilliseconds(200),
    BackoffType = DelayBackoffType.Exponential,
    UseJitter = true
};
retry.DisableForUnsafeHttpMethods();

var circuito = new HttpCircuitBreakerStrategyOptions
{
    FailureRatio = 0.5,
    MinimumThroughput = 4,
    SamplingDuration = TimeSpan.FromSeconds(10),
    BreakDuration = TimeSpan.FromSeconds(5)
};
```

- **Línea 1:** `var retry = new HttpRetryStrategyOptions` → crea opciones de retry HTTP con predicados transitorios de la integración.
- **Línea 2:** `{` → abre las opciones.
- **Línea 3:** `MaxRetryAttempts = 2,` → permite como máximo tres intentos en total.

- **Línea 4:** `Delay = TimeSpan.FromMilliseconds(200),` → establece demora base de 200 ms.
- **Línea 5:** `BackoffType = DelayBackoffType.Exponential,` → usa retroceso exponencial.
- **Línea 6:** `UseJitter = true` → añade jitter.
- **Línea 7:** `};` → cierra las opciones de retry.
- **Línea 8:** `retry.DisableForUnsafeHttpMethods();` → evita repetir métodos mutantes.
- **Línea 10:** `var circuito = new HttpCircuitBreakerStrategyOptions` → crea opciones de circuito HTTP.
- **Línea 11:** `{` → abre las opciones de circuito.
- **Línea 12:** `FailureRatio = 0.5,` → requiere una proporción de fallo del 50 % al evaluar muestras.
- **Línea 13:** `MinimumThroughput = 4,` → exige al menos cuatro ejecuciones en la ventana.
- **Línea 14:** `SamplingDuration = TimeSpan.FromSeconds(10),` → define una ventana de diez segundos.
- **Línea 15:** `BreakDuration = TimeSpan.FromSeconds(5)` → mantiene la apertura durante cinco segundos antes de permitir prueba.
- **Línea 16:** `};` → cierra las opciones.

La configuración se caracteriza por tres propiedades. La primera es que los parámetros se ajustan al contexto. La segunda es que la configuración se documenta. La tercera es que la configuración se versiona. Las tres propiedades permiten que la resiliencia sea coherente.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Los parámetros forman una política conjunta, no cuatro números independientes. La ventana de diez segundos exige cuatro muestras y ratio 0,5; apertura de cinco segundos regula cuándo intentar recuperación; el intento de un segundo y el total de seis segundos acotan espera y trabajo.

Ejemplo razonado. Con tráfico bajo, tres fallos en diez segundos no alcanzan MinimumThroughput = 4 y no abren el circuito. La petición sigue teniendo límites de retry y timeout. Con tráfico alto, varios intentos de distintas peticiones pueden completar el mínimo rápidamente.

Riesgo, alternativa y límite. Los valores son didácticos y deben documentarse por destino. No los presentes como recomendación universal para cualquier red. El jitter dificulta afirmar una duración exacta; la prueba debe centrarse en límites, número de envíos y eventos observables, sin sleeps largos y frágiles.

Comprobación y aprendizaje. Construye una tabla de parámetro, unidad, propósito y efecto si se duplica. Después verifica valores efectivos y comportamiento. Un cambio de apertura puede mejorar recuperación o generar nuevas oleadas; valorar ese compromiso es más importante que memorizar una constante.

Bloque 7 — Retry y idempotencia

El patrón Retry requiere que las operaciones sean idempotentes. Una operación idempotente mantiene el mismo efecto previsto al repetirse; las respuestas pueden variar, por ejemplo porque una lectura observa datos nuevos. Las operaciones de lectura son idempotentes. Las operaciones de escritura pueden no serlo. Cuando una operación de escritura no es idempotente, el reintento puede producir duplicados. Una escritura puede protegerse mediante un contrato de idempotencia: identidad estable, comprobación de payload y registro atómico del efecto o resultado. Incluir un identificador sin esa implementación no evita duplicados.

```
var retry = new HttpRetryStrategyOptions { MaxRetryAttempts = 2 };
retry.DisableForUnsafeHttpMethods();

using var response = await httpClient.GetAsync(
    "/api/materiales/CHAPA-3MM", ct);
```

- **Línea 1:** `var retry = new HttpRetryStrategyOptions { MaxRetryAttempts = 2 };` → configura el límite de retry antes de registrar la canalización.
- **Línea 2:** `retry.DisableForUnsafeHttpMethods();` → deshabilita repetición automática de métodos mutantes.
- **Línea 4:** `using var response = await httpClient.GetAsync(` → usa el cliente ya configurado y dispone su respuesta.
- **Línea 5:** `"/api/materiales/CHAPA-3MM", ct);` → propaga cancelación a una consulta de lectura; la ruta es ilustrativa.

La idempotencia se caracteriza por tres propiedades. La primera es que las operaciones idempotentes se pueden reintentar. La segunda es que repetir una operación con efectos exige un contrato de idempotencia implementado y verificado. La tercera es que la idempotencia se implementa en el servicio. Las tres propiedades permiten que el reintento sea seguro.

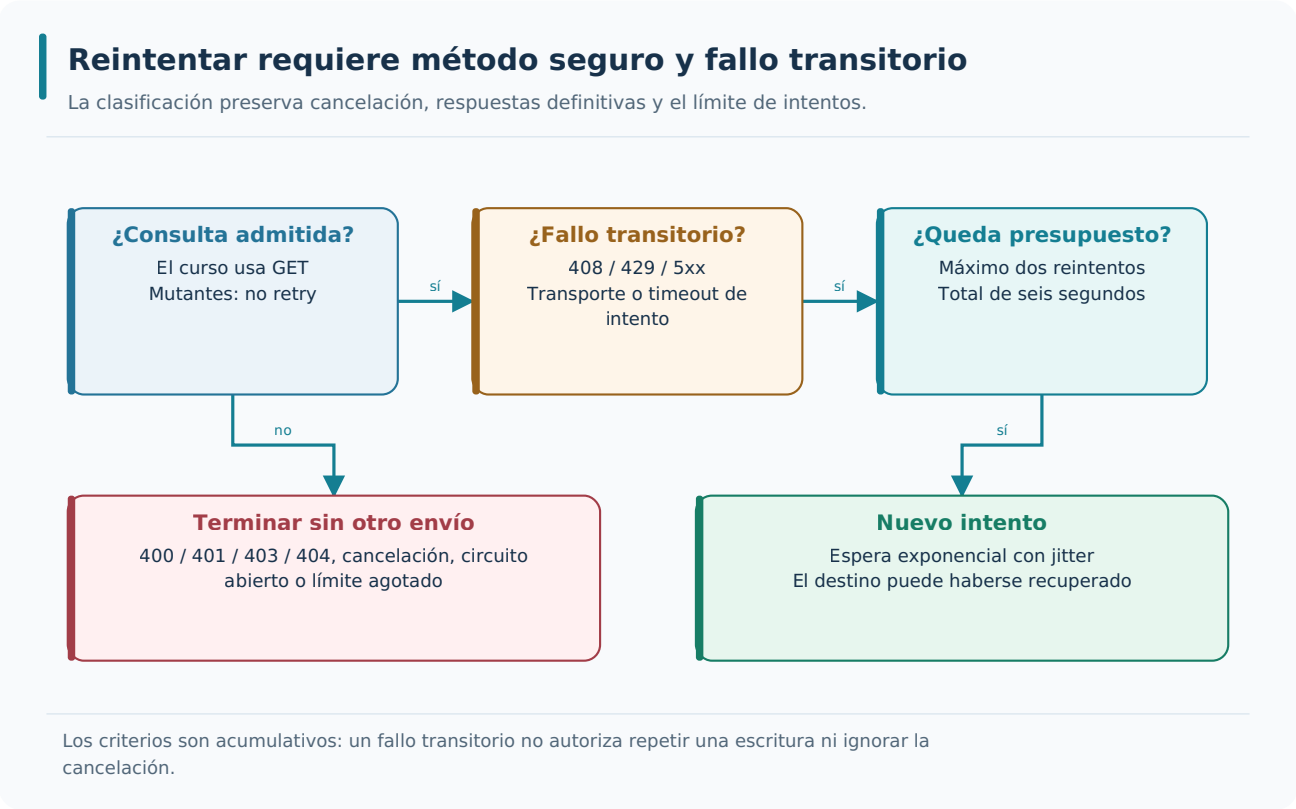
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Idempotencia describe el efecto previsto de repetir una operación, no exige que cada respuesta sea idéntica. GET puede devolver datos más recientes sin dejar de ser una lectura segura. El curso restringe retry a lectura; POST, PUT, PATCH y DELETE no se reintentan automáticamente.

Ejemplo razonado. Si un POST crea una orden y la respuesta se pierde, repetirlo podría crear otra. Un identificador en una cabecera no basta: el servidor tendría que guardar de forma atómica la identidad, la huella de la petición y el resultado, con reglas de concurrencia y retención.

Riesgo, alternativa y límite. Aunque HTTP define PUT y DELETE como idempotentes en su semántica, la política del módulo adopta una restricción más conservadora y no los reintenta. Tampoco se reutiliza el mismo HttpRequestMessage manualmente para varios envíos; la integración administra el transporte del cliente configurado.

Comprobación y aprendizaje. Ejecuta un fallo transitorio con GET y con cada método mutante. Compara el número de llamadas al transporte: solo la lectura puede aumentar. Relaciona este resultado con outbox e inbox de M6, donde la identidad estable y la deduplicación protegen efectos en una frontera distinta.



Interpretación del gráfico: Solo una operación admitida y un fallo transitorio pueden reintentarse. Método mutante, cancelación, 404 o circuito abierto detienen la repetición automática.

Bloque 8 — Circuit Breaker y fallback

El patrón Circuit Breaker puede combinarse con Fallback si existe una alternativa de negocio válida. El fallback ejecuta una alternativa explícita cuando el resultado o fallo cumple su criterio; un valor por defecto inventado puede ser incorrecto. El fallback permite que el sistema siga operando con funcionalidad degradada. En Polly v8 un fallback se define con AddFallback y sus opciones. El curso no activa un fallback de stock: traduce la indisponibilidad al contrato existente. El valor por defecto puede ser un valor constante o el resultado de una operación alternativa. La combinación de circuit breaker y fallback permite que el sistema sea tolerante a fallos.

```
try
{
    return await ConsultarMaterialRealAsync(codigo, ct);
}
catch (BrokenCircuitException ex)
{
    throw new InventarioNoDisponibleException("Inventario no está disponible temporalmente.", ex);
}
```

- **Línea 1:** `try` → delimita el ejemplo conceptual de adaptación.
- **Línea 2:** `{` → abre la consulta.
- **Línea 3:** `return await ConsultarMaterialRealAsync(codigo, ct);` → consulta mediante el cliente configurado y conserva cancelación.
- **Línea 4:** `}` → termina la región.
- **Línea 5:** `catch (BrokenCircuitException ex)` → reconoce que el circuito no permitió el envío.
- **Línea 6:** `{` → abre la traducción.

- **Línea 7:** `throw new InventarioNoDisponibleException("Inventario no está disponible temporalmente.", ex);` → comunica indisponibilidad conservando causa interna; no fabrica stock cero.
- **Línea 8:** `}` → cierra la traducción; la práctica incluye también timeout y cancelación.

La combinación se caracteriza por tres propiedades. La primera es que una alternativa debe tener semántica explícita y válida. La segunda es que el sistema sigue operando con funcionalidad degradada. La tercera es que el fallback se registra en los logs. Las tres propiedades permiten que el sistema sea tolerante a fallos.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Fallback es una alternativa explícita, no cualquier valor que evite una excepción. Para disponibilidad de material, devolver cero inventado confunde «no se pudo consultar» con «no hay existencias». El laboratorio conserva la indisponibilidad y su respuesta 503 cuando el circuito está abierto o se agota el tiempo.

Ejemplo razonado. Una interfaz podría mostrar un último dato conocido si existe una política clara de antigüedad y lo marca como obsoleto. Ese diseño sería distinto de usar un MaterialDto nuevo con cantidad cero. El módulo no incorpora una caché ni una reserva que hagan seguro continuar la planificación.

Riesgo, alternativa y límite. Una respuesta alternativa debe respetar invariantes del negocio. Convertir indisponibilidad en éxito silencioso puede ser más dañino que fallar. La excepción de circuito abierto pertenece a infraestructura y se traduce al contrato local sin exponer detalles de Polly al dominio.

Comprobación y aprendizaje. Detén Inventario hasta abrir el circuito y solicita material. Verifica indisponibilidad, ausencia de stock ficticio y falta de envíos mientras el circuito protege el destino. Después recupéralo y comprueba que un éxito real vuelve a ofrecer datos auténticos.

Bloque 9 — Buenas prácticas de resiliencia

Las buenas prácticas de resiliencia son cinco. La primera es aplicar timeout a todas las llamadas externas. La segunda es aplicar retry solo a operaciones idempotentes. La tercera es aplicar circuit breaker a los destinos que fallan con frecuencia. La cuarta es registrar los reintentos y las aperturas del circuito. La quinta es ajustar los parámetros según la operación y el destino. La aplicación de las cinco prácticas produce un sistema robusto y observable.

```
Buenas prácticas de resiliencia:

1. Timeout en todas las llamadas externas.
2. Retry solo en operaciones idempotentes.
3. Circuit breaker en destinos que fallan.
4. Registrar reintentos y aperturas.
5. Ajustar parámetros al contexto.
```

- **Línea 3:** `1. Timeout en todas las llamadas externas.` → primera práctica.
- **Línea 4:** `2. Retry solo en operaciones idempotentes.` → segunda práctica.
- **Línea 5:** `3. Circuit breaker en destinos que fallan.` → tercera práctica.
- **Línea 6:** `4. Registrar reintentos y aperturas.` → cuarta práctica.
- **Línea 7:** `5. Ajustar parámetros al contexto.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen un sistema robusto. La segunda es que la resiliencia es observable. La tercera es que los parámetros se ajustan al contexto. Las tres propiedades permiten que la resiliencia sea un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Prueba las decisiones de la política y sus límites. La observación útil incluye intentos, demoras, aperturas y resultado final con contexto seguro. No registres cuerpos ni mensajes libres de excepciones durante callbacks de resiliencia: las mismas reglas de privacidad siguen vigentes.

Ejemplo razonado. Una prueba controlada valida que 503 seguido de 200 se recupera, que 404 no se reintenta y que un circuito abierto no llama al transporte. Una prueba entre procesos detiene y recupera Inventario para comprobar que el registro del host y el cliente real usan esa política.

Riesgo, alternativa y límite. Los doubles son adecuados para secuencias exactas y reloj virtual, pero no prueban SQL, RabbitMQ ni conectividad entre procesos. A la inversa, una prueba de red con esperas fijas no es la mejor manera de verificar cada transición temporal de la biblioteca.

Comprobación y aprendizaje. Separa evidencia determinista de estrategia y evidencia de integración real. Comprueba que una cancelación voluntaria no se reintenta ni se transforma en 500. Conserva el fallo si falta infraestructura requerida: un test omitido no demuestra una recuperación que nunca ocurrió.

Bloque 10 — Aplicación al proyecto del curso y cierre del módulo 7

El proyecto del curso aplica los patrones de resiliencia en el módulo 7. Los clientes HTTP tipados se configuran con políticas de retry, circuit breaker y timeout. Los parámetros se ajustan según el destino. La combinación de patrones produce un sistema robusto. La validación conserva coherencia de políticas y añade secuencias controladas, límites de tiempo, métodos no reintentables, cancelación y caída/recuperación HTTP real. El módulo 7 cierra con el resumen del comportamiento observable y las evidencias del proyecto.

```
Aplicación al proyecto del curso:

Cliente HTTP      → políticas de retry, circuit breaker y timeout.
Parámetros        → ajustados por destino.
Registro          → reintentos y aperturas.
Validación        → coherencia, estrategias y HTTP real.
Cierre del módulo → resumen del estado del proyecto.
```

- **Línea 3:** `Cliente HTTP → políticas de retry, circuit breaker y timeout.` → políticas de retry, circuit breaker y timeout.
- **Línea 4:** `Parámetros → ajustados por destino.` → ajustados por destino.
- **Línea 5:** `Registro → reintentos y aperturas.` → reintentos y aperturas.
- **Línea 6:** `Validación → coherencia, estrategias y HTTP real.` → coherencia, estrategias y HTTP real.
- **Línea 7:** `Cierre del módulo → resumen del estado del proyecto.` → resumen del estado del proyecto.

El cierre del módulo 7 tiene dos consecuencias prácticas. La primera es que el proyecto tiene una base sólida de observabilidad y resiliencia. La segunda es que cada mecanismo tiene límites y pruebas documentados. La combinación de ambas permite que el sistema evolucione hacia la arquitectura de microservicios completa.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El cierre une observación, salud, errores y resiliencia sin cambiar las reglas del dominio. La consulta sigue siendo informativa y no reserva stock. Outbox e inbox mantienen su función de M6; los nuevos mecanismos hacen más visible y acotado el comportamiento de las fronteras técnicas.

Ejemplo razonado. Durante una caída de Inventario, Producción puede registrar la correlación, limitar intentos, abrir el circuito y responder indisponibilidad. Sus sondas informan del alcance de sus propias dependencias. Al recuperar Inventario, una prueba del circuito permite retomar lecturas reales.

Riesgo, alternativa y límite. Ningún catálogo de nombres ni test de coherencia acredita por sí solo ese recorrido. Tampoco disponer de estos mecanismos equivale a garantizar alta disponibilidad, entrega exactamente una vez o una plataforma completa de observabilidad. Las afirmaciones se limitan a lo implementado y comprobado.

Comprobación y aprendizaje. Como ejercicio de cierre, explica una operación correcta, una cancelada, una fallida y una recuperada, indicando evidencia, respuesta y capa responsable. Contrasta la teoría con las cuatro etapas ejecutables y sus pruebas. El módulo termina aquí, sin desarrollar contenidos de módulos posteriores.

Resumen rápido de la teoría

- La resiliencia es la capacidad de un sistema para recuperarse de fallos.
- Los patrones de resiliencia son retry, circuit breaker, timeout y bulkhead.
- El patrón Retry reintenta operaciones fallidas con un intervalo.
- El patrón Circuit Breaker corta el flujo cuando el destino falla de forma sostenida.
- Los patrones se combinan en una política completa.
- Polly es una biblioteca de .NET para la implementación de patrones de resiliencia.
- La configuración de políticas se realiza en el arranque.
- El retry requiere idempotencia en las operaciones.
- Un fallback necesita una alternativa válida; el curso responde indisponibilidad sin inventar stock.
- Las buenas prácticas son: timeout en todas las llamadas, retry solo en idempotentes, circuit breaker en destinos que fallan, registrar reintentos y aperturas, y ajustar parámetros al contexto.

Síntesis operativa del módulo

Pregunta	Mecanismo	Evidencia que aporta	Límite
¿Qué ocurrió en una petición?	Logs JSON y correlación	Servicio, resultado, duración e identidad técnica	No reconstruye eventos que nunca se registraron
¿Puede responder el proceso?	/health/live	Respuesta del host sin dependencias externas	No acredita SQL ni progreso de mensajes
¿Accede a dependencias necesarias?	/health/ready	Consulta SQL y conexión AMQP cuando corresponde	No prueba todos los casos de uso ni vaciado del outbox
¿Cómo se comunica un fallo inesperado?	IExceptionHandler y ProblemDetails	Error público uniforme y correlación	No puede reescribir una respuesta iniciada
¿Merece otro intento una lectura?	Retry acotado	Recuperación de fallos transitorios admitidos	No repite métodos mutantes ni cancelación
¿Conviene dejar de llamar al destino?	Circuito compartido	Rechazo rápido y prueba de recuperación	No repara la dependencia ni inventa datos

Bibliografía oficial y lecturas de ampliación

- Serilog.AspNetCore: integración y correspondencia de versiones.
- Logging en .NET: niveles, categorías y plantillas.
- Health checks en ASP.NET Core 8.
- Manejo de errores en ASP.NET Core 8.
- RFC 9457: Problem Details for HTTP APIs.
- Resiliencia HTTP en .NET.
- Polly v8: retry y circuit breaker.
- Polly: timeout y cancelación cooperativa.
- Kubernetes: liveness, readiness y startup.
- Dockerfile HEALTHCHECK.

Las referencias amplían los conceptos. El recorrido reproducible conserva las versiones fijadas en sus archivos de proyecto; no exige adoptar una API posterior que aparezca en la documentación general.